

Sveučilište Jurja Dobrile u Puli
Fakultet Informatike

MARTIN ČIKOR

Mobilna aplikacija za preporuku i naručivanje hrane uz primjenu velikih
jezičnih modela

Diplomski rad

Pula, Rujan, 2026. godine

Sveučilište Jurja Dobrile u Puli

Fakultet Informatike

MARTIN ČIKOR

Mobilna aplikacija za preporuku i naručivanje hrane uz primjenu velikih
jezičnih modela

Diplomski rad

JMBAG: 0307017377, izvanredni student

Studijski smjer: Informatika - nastavni smjer

Kolegij: Mobilne aplikacije

Znanstveno područje: Društvene znanosti

Znanstveno polje: Informacijske znanosti

Znanstvena grana: Informacijski sustavi i informatologija

Mentor: izv. prof. dr. sc. Nikola Tanković

Komentor: dr. sc. Robert Šajina

Pula, Rujan, 2026. godine



IZJAVA O AKADEMSKOJ ČESTITOSTI

Ja, dolje potpisani Martin Čikor, kandidat za magistra edukacije informatike ovime izjavljujem da je ovaj Diplomski rad rezultat isključivo mogega vlastitog rada, da se temelji na mojim istraživanjima te da se oslanja na objavljenu literaturu kao što to pokazuju korištene bilješke i bibliografija. Izjavljujem da niti jedan dio Diplomskog rada nije napisan na nedozvoljen način, odnosno da je prepisan iz necitiranog rada, te da nijedan dio rada ne krši bilo čija autorska prava. Izjavljujem, također, da nijedan dio rada nije iskorišten za neki drugi rad pri bilo kojoj drugoj visokoškolskoj, znanstvenoj ili radnoj ustanovi.

Student

Martin Čikor

Sadržaj

1	Uvod i motivacija.....	1
1.1	Analiza tržišta.....	2
2	Programske tehnologije i alati korišteni pri izradi.....	5
2.1	Razvojno okruženje Android Studio.....	5
2.2	Programski jezik Java.....	6
2.3	Sustav Firebase.....	7
2.4	Sustav za skeniranje QR kodova Google Code Scanner	10
2.5	OpenAI API i veliki jezični modeli	10
2.6	Mobilna platforma Android	12
2.7	Operacijski sustav Android.....	12
2.8	Arhitektura Android operacijskog sustava.....	13
2.9	Životni ciklus Android aktivnosti.....	15
3	Implementacija mobilne aplikacije	17
3.1	Dijagram slučajeva korištenja.....	17
3.2	Arhitektura i organizacija mobilne aplikacije.....	18
3.3	Početna prijava korisnika	20
3.4	Skeniranje QR koda.....	22
3.5	Prikaz digitalnog jelovnika.....	23
3.6	Prilagodba proizvoda i dodavanje u narudžbu	23
3.7	Pregled narudžbe	26
3.8	AI asistent u mobilnoj aplikaciji.....	27
3.9	Potvrda narudžbe	30
3.10	Pregled aktivnih narudžbi.....	32
3.11	Praćenje narudžbe.....	34
3.12	Pristup kuhinjskom dijelu aplikacije	35
3.13	Ažuriranje statusa narudžbe.....	38
4	Implementacija poslužiteljskog dijela	40
4.1	Organizacija poslužiteljske logike	41
4.2	Komunikacija između komponenti sustava	42
4.3	Komunikacija s modelima umjetne inteligencije	45
4.4	Planiranje razgovora i prepoznavanje korisničke namjere	45
4.5	Normalizacija i obrada strukturiranog plana razgovora	47
4.6	Obrada korisničkih zahtjeva i generiranje preporuka	48
4.7	Obrada korisničkog zahtjeva	50
4.8	Projektiranje prompta.....	52
4.9	Odabir modela GPT-5-mini	53
4.10	Ograničenja primijenjenog rješenja	57

4.11	Potrošnja tokena i procjena troškova.....	58
5	Testiranje sustava.....	60
5.1	Funkcionalno testiranje mobilne aplikacije.....	60
5.2	Evaluacija AI asistenta.....	62
6	Zaključak.....	65
	Literatura.....	67
	Dodatak A – Promptovi korišteni za AI asistenta.....	69
	Popis slika.....	71
	Popis isječaka programskog koda.....	72
	Popis tablica.....	73
	Sažetak.....	74
	Abstract.....	75

1 Uvod i motivacija

Razvoj mobilnih tehnologija i umjetne inteligencije značajno je promijenio način korištenja digitalnih usluga u ugostiteljstvu. Iako mnogi objekti nude digitalne jelovnike i naručivanje hrane, većina rješenja temelji se na ručnom pregledavanju ponude, bez dodatne pomoći pri odabiru. Velik broj dostupnih proizvoda korisnicima može otežati i usporiti donošenje odluke.

Veliki jezični modeli (engl. Large Language Models – LLM) omogućuju prirodnu komunikaciju s korisnikom, razumijevanje njegovih zahtjeva i generiranje preporuka na temelju dostupne ponude, preferencija i prethodnih narudžbi. Primjena velikih jezičnih modela može unaprijediti preporučene sustave omogućavanjem prirodnije interakcije i boljeg iskorištavanja kontekstualnih informacija [5].

Motivacija za izradu rada bila je razvoj mobilne aplikacije koja povezuje klasično digitalno naručivanje s mogućnostima velikih jezičnih modela. Razvijeno rješenje omogućuje anonimnu prijavu, identifikaciju stola skeniranjem QR koda, pregled jelovnika, prilagodbu narudžbe i komunikaciju s AI asistentom koji odgovara na pitanja i preporučuje proizvode.

Razvojni problem rada odnosi se na pitanje kako primjenom velikih jezičnih modela unaprijediti digitalno naručivanje hrane i omogućiti prirodniju interakciju tijekom odabira proizvoda. Cilj rada je prikazati razvoj mobilne aplikacije, od analize postojećih rješenja i odabira tehnologija do implementacije, integracije s platformom Firebase i AI modelom te testiranja funkcionalnosti sustava.

Rad je organiziran u šest poglavlja. Nakon uvoda i analize postojećih rješenja slijedi pregled programskih tehnologija i alata korištenih pri razvoju sustava. Treće poglavlje opisuje implementaciju mobilne aplikacije, uključujući glavne funkcionalnosti povezane s prijavom korisnika, identifikacijom stola, pregledom jelovnika, naručivanjem i praćenjem narudžbi. U četvrtom poglavlju prikazana je implementacija poslužiteljskog dijela sustava, komunikacija s velikim jezičnim modelom te obrada korisničkih zahtjeva i generiranje preporuka. Peto poglavlje obuhvaća funkcionalno testiranje mobilne aplikacije i evaluaciju AI asistenta, dok su u šestom poglavlju izneseni zaključci rada, ograničenja razvijenog sustava i mogućnosti budućih nadogradnji.

1.1 Analiza tržišta

Analiza tržišta predstavlja proces prikupljanja i proučavanja informacija o postojećim proizvodima, korisnicima i konkurenciji unutar određenog tržišnog segmenta. Njezin je cilj razumjeti potrebe korisnika, identificirati prednosti i nedostatke postojećih rješenja te prepoznati mogućnosti za razvoj novih funkcionalnosti koje mogu unaprijediti korisničko iskustvo.

Prije razvoja mobilne aplikacije provedena je analiza postojećih rješenja za digitalno naručivanje hrane i pića. Posebna pažnja posvećena je aplikacijama koje omogućuju pregled digitalnih jelovnika, naručivanje hrane i pića te primjenu umjetne inteligencije u procesu preporuke proizvoda. Na temelju provedene analize identificirane su funkcionalnosti koje korisnici očekuju od suvremenih aplikacija za naručivanje, kao i područja u kojima postoji prostor za daljnja poboljšanja.

U okviru analize razmotrena su četiri postojeća rješenja koja se razlikuju prema području primjene i načinu korištenja umjetne inteligencije:

- Dobartek je hrvatska platforma za online naručivanje hrane koja korisnicima omogućuje pregled restorana, digitalnih jelovnika i jednostavno naručivanje hrane za dostavu ili osobno preuzimanje. Sustav omogućuje pretraživanje dostupnih restorana i njihove ponude te pojednostavljuje proces odabira i naručivanja hrane. Korisničko sučelje temelji se na pregledavanju kategorija i proizvoda, bez mogućnosti vođenja prirodnog razgovora s korisnikom ili pružanja personaliziranih preporuka temeljenih na umjetnoj inteligenciji [1].
- FoodApp je hrvatska platforma za digitalizaciju ugostiteljskog poslovanja koja omogućuje pregled digitalnih jelovnika, online naručivanje hrane i pića te povezivanje korisnika s ugostiteljskim objektima. Interakcija se temelji na standardnom grafičkom korisničkom sučelju, bez konverzacijskog AI asistenta [2].
- me&u je međunarodna platforma namijenjena digitalnom naručivanju hrane i pića unutar ugostiteljskih objekata. Korisnik nakon dolaska u restoran skenira QR kod postavljen na stolu, čime dobiva pristup digitalnom jelovniku te mogućnost naručivanja i elektroničkog plaćanja bez potrebe za pozivanjem konobara. Platforma koristi umjetnu inteligenciju za personalizaciju ponude i preporuku dodatnih proizvoda, čime nastoji unaprijediti korisničko iskustvo i povećati vrijednost narudžbe. Unatoč tome, komunikacija s korisnikom i dalje se

temelji na klasičnom grafičkom korisničkom sučelju, bez mogućnosti vođenja konverzacijskog dijaloga prirodnim jezikom [3].

•DoorDash je jedna od vodećih svjetskih platformi za naručivanje hrane koja je uvela generativnu umjetnu inteligenciju za pretraživanje jelovnika i generiranje personaliziranih preporuka. Korisnicima omogućuje opisivanje željenog obroka prirodnim jezikom, nakon čega sustav analizira zahtjev i predlaže odgovarajuće proizvode. Primjena umjetne inteligencije usmjerena je na olakšavanje pretraživanja ponude i donošenja odluke o narudžbi. Iako predstavlja primjer suvremene primjene velikih jezičnih modela u području naručivanja hrane, osnovna namjena platforme i dalje je organizacija dostave hrane, a ne naručivanje unutar ugostiteljskog objekta [4].

Funkcionalnost	Dobartek	FoodApp	me&u	DoorDash AI	Razvijena aplikacija
Digitalni jelovnik	✓	✓	✓	✓	✓
QR identifikacija stola	X	✓	✓	X	✓
Naručivanje hrane i pića	✓	✓	✓	✓	✓
Personalizirane preporuke	X	X	✓	✓	✓
AI asistent	X	X	X	✓	✓
Razgovor prirodnim jezikom	X	X	X	✓	✓
Naručivanje unutar ugostiteljskog objekta	X	✓	✓	X	✓

Tablica 1. Usporedba funkcionalnosti analiziranih aplikacija.

Iz tablice 1 vidljivo je da sva analizirana rješenja omogućuju pregled digitalnog jelovnika i digitalno naručivanje hrane, što predstavlja osnovne funkcionalnosti suvremenih aplikacija u ugostiteljstvu. Međutim, većina analiziranih sustava temelji se na klasičnom grafičkom korisničkom sučelju, pri čemu korisnik samostalno pretražuje kategorije proizvoda, pregledava informacije o jelima i donosi odluku o narudžbi bez dodatne pomoći sustava. Takav način interakcije zahtijeva aktivno pretraživanje ponude te može biti manje praktičan u situacijama kada korisnik nije upoznat s jelovnikom ili nije siguran što želi naručiti.

Primjena umjetne inteligencije prisutna je samo u pojedinim novijim rješenjima, gdje se koristi za personalizirane preporuke, pretraživanje jelovnika prirodnim jezikom ili predlaganje dodatnih proizvoda. Ipak, u većini slučajeva umjetna inteligencija predstavlja pomoćnu funkcionalnost, dok se cjelokupan proces naručivanja i dalje temelji na tradicionalnom grafičkom korisničkom sučelju.

Analizirana rješenja razlikuju se i prema području primjene. Dobartek i DoorDash prvenstveno su usmjereni na naručivanje hrane za dostavu ili osobno preuzimanje, dok su FoodApp i me&u prilagođeni digitalnom naručivanju unutar ugostiteljskih objekata. Iako pojedina rješenja koriste umjetnu inteligenciju za personalizaciju ponude ili preporuku proizvoda, niti jedno od analiziranih sustava ne objedinjuje QR identifikaciju stola, digitalni jelovnik i konverzijskog AI asistenta u jedinstven proces naručivanja.

Razvijena aplikacija razlikuje se od analiziranih sustava integracijom AI asistenta temeljenog na velikom jezičnom modelu, koji korisniku omogućuje postavljanje pitanja o jelovniku, dobivanje personaliziranih preporuka te pomoć pri sastavljanju narudžbe kroz prirodan razgovor. Umjesto samostalnog pretraživanja velikog broja proizvoda, korisnik može opisati svoje želje prirodnim jezikom, primjerice zatražiti vegetarijansko jelo, preporuku pića uz odabrani obrok ili pomoć pri sastavljanju kompletnog obroka. Takav način interakcije pojednostavljuje proces naručivanja i pruža prirodnije i personaliziranije korisničko iskustvo. Razvijena aplikacija time objedinjuje funkcionalnosti digitalnog jelovnika, naručivanja i umjetne inteligencije u jedinstveno rješenje namijenjeno korištenju unutar ugostiteljskih objekata.

2 Programske tehnologije i alati korišteni pri izradi

Razvoj mobilne aplikacije zahtijevao je korištenje više međusobno povezanih tehnologija koje zajedno omogućuju razvoj korisničkog sučelja, pohranu podataka, komunikaciju s poslužiteljskom logikom te implementaciju funkcionalnosti temeljenih na umjetnoj inteligenciji. U ovom poglavlju opisane su tehnologije korištene tijekom razvoja sustava te razlozi njihova odabira.

2.1 Razvojno okruženje Android Studio

Android Studio službeno je integrirano razvojno okruženje (engl. Integrated Development Environment – IDE) za razvoj Android aplikacija, koje razvija i održava Google. Temelji se na razvojnom okruženju IntelliJ IDEA te programerima pruža sve potrebne alate za razvoj, testiranje, otklanjanje pogrešaka i objavu mobilnih aplikacija [6].

Jedna od najvećih prednosti Android Studija jest objedinjavanje svih faza razvoja aplikacije unutar jednog okruženja. Omogućuje uređivanje izvornog koda, vizualno oblikovanje korisničkog sučelja, upravljanje projektom s pomoću sustava Gradle, pokretanje aplikacije na emulatoru ili fizičkom uređaju te analizu performansi tijekom izvođenja aplikacije [6].

U okviru ovog rada Android Studio korišten je za razvoj cjelokupne mobilne aplikacije, implementaciju korisničkog sučelja, integraciju vanjskih biblioteka te povezivanje s Firebase platformom i OpenAI API-jem. Razvojno okruženje omogućilo je jednostavno testiranje funkcionalnosti poput prijave korisnika, skeniranja QR koda, komunikacije s bazom podataka i rada AI asistenta, čime je značajno ubrzan proces razvoja aplikacije.

Android Studio odabran je kao razvojno okruženje zbog službene podrške za razvoj Android aplikacija te integracije svih alata potrebnih za razvoj, testiranje i otklanjanje pogrešaka unutar jednog okruženja. Alternativna razvojna okruženja, poput Visual Studio Codea ili Eclipsea, mogu se koristiti za razvoj Android aplikacija, no zahtijevaju dodatnu konfiguraciju i ne pružaju jednaku razinu integracije s Android SDK-om, emulatorom i alatima za profiliranje aplikacije. Zbog toga je Android Studio predstavljao najprikladnije razvojno okruženje za razvoj aplikacije.

2.2 Programski jezik Java

Java je objektno orijentirani programski jezik koji je razvila tvrtka Sun Microsystems, a danas ga razvija i održava tvrtka Oracle. Zahvaljujući prenosivosti, pouzdanosti, jasno definiranoj objektno orijentiranoj strukturi i velikoj zajednici programera, Java se dugi niz godina intenzivno koristi za razvoj različitih vrsta aplikacija, uključujući aplikacije za operacijski sustav Android [7].

U klasičnim se okruženjima izvorni kod napisan u Javi prevodi u međukod koji se izvršava unutar Java virtualnog stroja (engl. Java Virtual Machine – JVM). Time se omogućuje izvođenje istoga programskog koda na različitim operacijskim sustavima koji podržavaju odgovarajuću implementaciju JVM-a. Pri razvoju Android aplikacija Java međukod pretvara se u DEX (engl. Dalvik Executable) format prilagođen Android platformi. DEX datoteke izvršavaju se unutar okruženja Android Runtime (ART), koje predstavlja upravljano izvršno okruženje Android sustava [8].

Jedna od glavnih prednosti Jave jest objektno orijentirani pristup programiranju, koji omogućuje modularnu organizaciju aplikacije, jednostavnije održavanje i ponovnu uporabu programskog koda. Jezik također pruža bogat skup ugrađenih biblioteka te podršku za velik broj vanjskih razvojnih alata i programskih okvira [9].

U ovom radu Java je korištena za implementaciju cjelokupne logike mobilne aplikacije. Njome su razvijene aktivnosti aplikacije, upravljanje korisničkim sučeljem, obrada korisničkih događaja, komunikacija s Firebase servisima te integracija s ostalim korištenim tehnologijama. Java je također korištena za obradu podataka tijekom skeniranja QR koda, upravljanje narudžbama te komunikaciju s uslugom Firebase Functions, koja omogućuje korištenje AI asistenta.

Iako Google za razvoj novih Android aplikacija preporučuje programski jezik Kotlin, Java je i dalje u širokoj uporabi te predstavlja stabilno i pouzdano rješenje za razvoj mobilnih aplikacija. Pri izradi ovog rada Java je odabrana zbog dobre podrške korištenih biblioteka, jednostavnije implementacije planiranih funkcionalnosti te prethodnog iskustva autora s navedenim programskim jezikom. Budući da razvijena aplikacija ne zahtijeva značajke specifične za Kotlin, odabir Jave nije predstavljao ograničenje, već je omogućio jednostavniji razvoj i održavanje programskog koda.

2.3 Sustav Firebase

Firebase je platforma za razvoj aplikacija u oblaku (engl. Backend-as-a-Service – BaaS) koju razvija i održava Google. Programerima pruža skup gotovih usluga koje pojednostavljaju razvoj mobilnih i web aplikacija, bez potrebe za implementacijom i održavanjem vlastitog poslužitelja. Platforma objedinjuje različite servise, poput autentifikacije korisnika, baza podataka u stvarnom vremenu, pohrane datoteka, analitike i izvršavanja poslužiteljske logike [10].

Jedna od glavnih prednosti Firebase platforme jest jednostavna integracija s Android aplikacijama te mogućnost razvoja skalabilnih rješenja uz minimalnu konfiguraciju infrastrukture. Firebase automatski sinkronizira promjene između poslužitelja i povezanih klijenata, pa svi klijenti prikazuju ažurne podatke u stvarnom vremenu [10].

U okviru ovog rada Firebase predstavlja središnji dio sustava te omogućuje komunikaciju između mobilne aplikacije, baze podataka i AI asistenta. Platforma je korištena za anonimnu autentifikaciju korisnika, pohranu podataka o jelovniku, stolovima i narudžbama te izvršavanje poslužiteljske logike zadužene za obradu korisničkih zahtjeva i komunikaciju s velikim jezičnim modelom. U tu svrhu korištena su tri Firebase servisa: Firebase Authentication, Firebase Realtime Database i Firebase Functions.

Firebase Authentication omogućuje implementaciju autentifikacije korisnika unutar mobilnih i web aplikacija. Servis podržava različite načine prijave, uključujući prijavu putem elektroničke pošte i lozinke, Google korisničkog računa, Facebooka, Apple ID-ja, telefonskog broja te anonimnu autentifikaciju. Firebase pritom preuzima brigu o sigurnosti korisničkih podataka, upravljanju korisničkim identitetom i izdavanju autentifikacijskih tokena [11].

U razvijenoj aplikaciji korištena je anonimna autentifikacija. Takav način prijave omogućuje korisniku korištenje aplikacije bez potrebe za registracijom ili prijavom korisničkim računom. Prilikom prvog pokretanja aplikacije automatski se kreira anonimni korisnički račun kojem Firebase dodjeljuje jedinstveni identifikator (UID). Taj identifikator koristi se za razlikovanje korisnika te povezivanje njihovih podataka, razgovora s AI asistentom i povijesti narudžbi.

Odabir anonimne autentifikacije pokazao se prikladnim zbog načina korištenja aplikacije. Budući da je aplikacija namijenjena gostima ugostiteljskog objekta koji nakon dolaska skeniraju QR kod sa stola i odmah započinju proces naručivanja, zahtijevanje

registracije ili prijave predstavljalo bi nepotrebno usporavanje i narušilo korisničko iskustvo. Anonimna autentifikacija omogućuje brz pristup aplikaciji uz zadržavanje mogućnosti jedinstvene identifikacije svakog korisnika tijekom korištenja sustava. Nakon uspješne autentifikacije korisnik unosi nadimak koji se lokalno pohranjuje na uređaju te se koristi tijekom komunikacije s AI asistentom i prikaza pojedinih informacija unutar aplikacije.

Firestore Realtime Database je NoSQL baza podataka u oblaku koja omogućuje pohranu i sinkronizaciju podataka u stvarnom vremenu. Podaci se spremaju u obliku JSON strukture, što omogućuje jednostavno dohvaćanje i ažuriranje podataka iz mobilne aplikacije. Za razliku od relacijskih baza podataka, Firestore Realtime Database ne koristi tablice i SQL upite, već podatke organizira hijerarhijski, u obliku čvorova [12].

U okviru ovog rada Firestore Realtime Database koristi se kao središnje mjesto za pohranu podataka potrebnih za rad aplikacije. U bazi se nalaze podaci o stolovima, QR tokenima, jelovniku, hrani, pićima, modifikacijama proizvoda i narudžbama. Prilikom skeniranja QR koda aplikacija provjerava postoji li odgovarajući zapis u bazi pod čvorom stolova te je li stol aktivan. Nakon uspješne provjere korisniku se omogućuje pristup digitalnom jelovniku.

Firestore Realtime Database također se koristi za pohranu i dohvat narudžbi. Nakon što korisnik potvrdi narudžbu, ona se sprema u bazu i postaje dostupna ostalim dijelovima sustava, primjerice kuhinjskom prikazu. Osim toga, podaci iz baze koriste se u Firestore Functions servisu prilikom komunikacije s AI asistentom, gdje se dohvaćaju dostupni artikli, prethodne narudžbe korisnika i druge informacije potrebne za generiranje preporuka.

Firestore Realtime Database odabrana je zbog potrebe za trenutačnom sinkronizacijom podataka između više korisnika i uređaja. Alternativno rješenje predstavlja Cloud Firestore, koji nudi naprednije mogućnosti upita i veću fleksibilnost pri organizaciji podataka, ali za potrebe ovog rada hijerarhijska struktura Realtime Database baze pokazala se jednostavnijom za implementaciju. Budući da aplikacija koristi relativno jednostavan model podataka i zahtijeva brzo ažuriranje narudžbi u stvarnom vremenu, Firestore Realtime Database predstavljala je prikladan izbor.

Firestore Functions usluga je koja omogućuje izvršavanje poslužiteljske logike bez potrebe za razvojem i održavanjem vlastitog poslužitelja. Takav pristup poznat je kao računalstvo bez poslužitelja (engl. serverless), pri čemu se programski kod izvršava kada ga pozove aplikacija ili neki drugi Firestore servis. Tako se programer može usredotočiti na

implementaciju poslovne logike, dok Google upravlja infrastrukturom te osigurava skaliranje i dostupnost sustava [13].

U razvijenom sustavu Firebase Functions koristi se kao središnji dio poslužiteljske logike zadužen za komunikaciju između mobilne aplikacije, Firebase Realtime Database baze podataka i velikog jezičnog modela. Mobilna aplikacija korisničke zahtjeve ne šalje izravno prema OpenAI API-ju, već ih prosljeđuje Firebase funkciji koja provodi potrebnu obradu prije generiranja odgovora.

Prilikom zaprimanja korisničkog zahtjeva funkcija najprije dohvaća podatke iz Firebase Realtime Database baze, uključujući informacije o jelovniku, pićima, prethodnim narudžbama korisnika i ostalim podacima potrebnima za obradu zahtjeva. Nakon toga analiziraju se korisnikova poruka i kontekst razgovora kako bi se utvrdila namjera korisnika. Ovisno o vrsti zahtjeva, funkcija može dohvatiti prethodne narudžbe, preporučiti proizvode, odgovoriti na pitanja o jelovniku ili pripremiti podatke koji će biti poslani velikom jezičnom modelu.

Važna prednost ovakvog pristupa jest što se poslovna logika nalazi na poslužiteljskoj strani sustava. Time se osigurava da mobilna aplikacija ne sadrži osjetljive podatke, poput API ključeva za pristup vanjskim servisima, niti izravno komunicira s OpenAI API-jem. Firebase Functions izvršava sigurnosno osjetljive operacije na poslužiteljskoj strani, štiti pristupne podatke i pojednostavljuje održavanje sustava.

Firebase Functions odabrane su kako bi se poslovna logika izdvojila iz mobilne aplikacije i izvršavala na poslužiteljskoj strani. Takav pristup omogućuje sigurnu pohranu API ključeva, jednostavnije održavanje sustava i dodavanje novih funkcionalnosti bez potrebe za objavom nove verzije mobilne aplikacije. U usporedbi s razvojem vlastitog poslužitelja, serverless pristup smanjuje potrebu za izravnim upravljanjem infrastrukturom te omogućuje automatsko skaliranje računalnih resursa ovisno o opterećenju sustava [14].

Firebase je stoga odabran kao zajednička platforma za navedene funkcionalnosti zbog jednostavne integracije s Android aplikacijama i dostupnosti servisa potrebnih za razvoj sustava. Alternativni pristup bio bi razvoj vlastitog poslužiteljskog sustava, primjerice korištenjem Spring Boota ili Node.js-a uz zasebnu bazu podataka. Takvo rješenje pružilo bi veću kontrolu nad infrastrukturom, ali bi zahtijevalo dodatno vrijeme za razvoj, konfiguraciju i održavanje. Za potrebe ovog rada Firebase je omogućio jednostavniju implementaciju autentifikacije, pohrane podataka i poslužiteljske logike unutar jedinstvene platforme.

2.4 Sustav za skeniranje QR kodova Google Code Scanner

Google Code Scanner API omogućuje jednostavnu implementaciju skeniranja barkodova i QR kodova u Android aplikacijama. API je dio Googleovih razvojnih alata povezanih s platformom ML Kit, a za skeniranje koristi usluge Google Play. Programeru pruža gotovo korisničko sučelje za skeniranje, zbog čega nije potrebno samostalno implementirati prikaz kamere, obradu slike ni upravljanje dopuštenjima za pristup kameri [15].

U ovom radu Google Code Scanner API korišten je za skeniranje QR kodova postavljenih na stolovima ugostiteljskog objekta. Skener je konfiguriran tako da prepoznaje isključivo QR kodove. Nakon uspješnog skeniranja aplikacija dohvaća tekstualnu vrijednost QR koda i provodi njezinu validaciju prije nastavka rada.

QR kodovi u razvijenom sustavu služe za identifikaciju stola unutar ugostiteljskog objekta. Svaki stol ima jedinstveni QR kod koji sadrži identifikacijski token. Nakon skeniranja aplikacija provjerava postoji li odgovarajući zapis u bazi Firebase Realtime Database te je li stol aktivan. Ako je provjera uspješna, korisniku se prikazuje digitalni jelovnik i omogućuje nastavak procesa naručivanja.

Google Code Scanner API odabran je zbog jednostavne integracije s Android platformom i činjenice da pruža gotovo sučelje za skeniranje. Alternativna rješenja, poput biblioteke ZXing ili API-ja ML Kit Barcode Scanning, omogućuju veću kontrolu nad prikazom kamere i obradom rezultata, ali zahtijevaju dodatnu implementaciju. Budući da razvijena aplikacija zahtijeva jednostavno skeniranje QR koda bez prilagođenog prikaza kamere, Google Code Scanner predstavljao je prikladno rješenje.

2.5 OpenAI API i veliki jezični modeli

Veliki jezični modeli (engl. Large Language Models – LLM) predstavljaju jednu od najznačajnijih primjena umjetne inteligencije posljednjih godina. Riječ je o modelima strojnog učenja treniranim na velikim količinama tekstualnih podataka, zbog čega su sposobni obrađivati prirodni jezik i generirati sadržajno povezane tekstualne odgovore. Suvremeni veliki jezični modeli uglavnom se temelje na transformer arhitekturi, koja koristi mehanizam pažnje za obradu odnosa između elemenata ulaznog teksta [16].

Za razliku od klasičnih chatbot sustava koji se temelje na unaprijed definiranim pravilima i scenarijima razgovora, veliki jezični modeli mogu analizirati korisnički zahtjev u kontekstu prethodnih poruka i generirati odgovore prilagođene pojedinoj situaciji. Zbog toga su prikladni za razvoj konverzacijskih asistenata i preporučenih sustava koji korisniku omogućuju prirodni način interakcije [5].

U ovom radu za komunikaciju s velikim jezičnim modelom korišten je OpenAI API. OpenAI API omogućuje pristup modelima umjetne inteligencije putem programskog sučelja (engl. Application Programming Interface – API), čime se omogućuje njihova integracija u postojeće aplikacije. Komunikacija s modelom ostvarena je korištenjem Responses API-ja, koji omogućuje slanje korisničkog zahtjeva i dohvaćanje generiranog odgovora [17].

U trenutku razvoja sustava korišten je model GPT-5-mini [18]. Model je odabran kao prikladno rješenje za potrebe razvijenog AI asistenta, koji je namijenjen razumijevanju korisničkih zahtjeva, odgovaranju na pitanja o jelovniku, preporučivanju jela i pića te pomoći pri sastavljanju narudžbe. Budući da se poslovna pravila i provjera podataka izvršavaju na poslužiteljskoj strani, veliki jezični model prvenstveno se koristi za razumijevanje prirodnog jezika i oblikovanje strukturiranog plana obrade.

Poziv prema OpenAI API-ju ne izvršava se izravno iz Android aplikacije, već unutar Firebase Functions servisa. Takav pristup omogućuje da API ključ ostane pohranjen na poslužiteljskoj strani sustava te nije dostupan krajnjim korisnicima aplikacije, čime se povećava sigurnost sustava.

Prije slanja zahtjeva modelu Firebase Functions prikuplja informacije potrebne za generiranje odgovora. Uključivanje relevantnog konteksta, preciznih uputa i ograničenja u prompt može povećati dosljednost odgovora i olakšati njihovu daljnju programsku obradu [19]. Ovisno o korisničkom zahtjevu, funkcija dohvaća podatke o jelovniku, dostupnim jelima i pićima, prethodnim narudžbama korisnika, trenutačnom sadržaju košarice te drugim informacijama koje mogu biti važne za razgovor. Na temelju tih podataka oblikuje se odgovarajući kontekst koji se zatim prosljeđuje velikom jezičnom modelu.

U razvijenom sustavu veliki jezični model ima ulogu inteligentnog asistenta za naručivanje hrane. Korisnik može prirodnim jezikom zatražiti preporuku jela, postaviti pitanje o pojedinom proizvodu, zatražiti prijedlog kompletnog obroka, ponoviti prethodnu narudžbu ili dodati proizvode u košaricu bez ručnog pretraživanja digitalnog jelovnika.

Preporučeni sustavi imaju cilj korisniku olakšati pronalaženje relevantnih proizvoda ili sadržaja na temelju dostupnih informacija o korisniku, stavkama i kontekstu korištenja [20]. U razvijenom sustavu takav je pristup proširen primjenom velikog jezičnog modela, koji omogućuje obradu zahtjeva izraženih prirodnim jezikom i njihovo povezivanje s dostupnim proizvodima iz digitalnog jelovnika. Za razliku od klasičnih aplikacija za naručivanje hrane, razvijena aplikacija omogućuje prirodnu komunikaciju s AI asistentom koji razumije korisnikove zahtjeve i pruža personalizirane preporuke tijekom procesa naručivanja. Takva komunikacija korisniku olakšava odabir hrane i pića.

2.6 Mobilna platforma Android

Android je mobilna platforma koju razvija tvrtka Google te predstavlja jednu od najraširenijih platformi za razvoj mobilnih aplikacija. Zahvaljujući otvorenoj arhitekturi, bogatom razvojnem okruženju i velikom broju dostupnih programskih sučelja omogućuje razvoj modernih, sigurnih i funkcionalnih aplikacija za različite vrste mobilnih uređaja [21].

U okviru ovog rada Android je odabran kao razvojna platforma zbog izravne podrške za Googleove razvojne alate i servise korištene tijekom razvoja sustava, uključujući Firebase i Google ML Kit. Alternativna višepatformska rješenja (engl. cross-platform), poput Fluttera ili React Nativea, omogućuju razvoj aplikacija za više operacijskih sustava iz jedinstvene baze izvornog koda, ali zahtijevaju dodatni sloj apstrakcije pri pristupu pojedinim funkcionalnostima uređaja. Budući da je cilj ovog rada bio razviti aplikaciju isključivo za Android uređaje uz jednostavnu integraciju s Googleovim tehnologijama, nativni razvoj na Android platformi predstavljao je najprikladniji odabir.

2.7 Operacijski sustav Android

Android operacijski sustav (engl. *Android Operating System* – Android OS) temelji se na Linux jezgri te je namijenjen radu na različitim vrstama mobilnih uređaja, uključujući pametne telefone, tablete, pametne satove i druge uređaje. Njegova glavna uloga je upravljanje hardverskim resursima uređaja, izvršavanje aplikacija te omogućavanje komunikacije između korisničkih aplikacija i hardverskih komponenti uređaja [21].

Android programerima pruža velik broj programskih sučelja (engl. *Application Programming Interface* – API) za pristup funkcionalnostima poput kamere, mrežne komunikacije, senzora, lokalne pohrane i korisničkog sučelja. Time se pojednostavljuje

razvoj aplikacija jer programer ne mora izravno upravljati hardverskim komponentama uređaja, već koristi unaprijed definirana programska sučelja koja osigurava operacijski sustav [21].

U okviru ovog rada Android operacijski sustav omogućuje izvršavanje mobilne aplikacije, pristup kameri uređaja za skeniranje QR kodova, komunikaciju s Firebase servisima te prikaz korisničkog sučelja putem kojeg korisnik pregledava digitalni jelovnik, komunicira s AI asistentom i izrađuje narudžbu.

2.8 Arhitektura Android operacijskog sustava

Arhitektura Android operacijskog sustava sastoji se od više međusobno povezanih slojeva i komponenti koji omogućuju stabilan i učinkovit rad mobilnih aplikacija. Svaki dio arhitekture ima određenu ulogu u povezivanju aplikacijskog softvera s temeljnim resursima sustava, čime se programerima olakšava razvoj aplikacija bez potrebe za izravnom komunikacijom s hardverom uređaja. Temelj sustava čini Linux jezgra, iznad koje se nalaze native biblioteke i Android Runtime (ART), dok aplikacijski okvir (engl. Application Framework) pruža programska sučelja i sistemske usluge koje koriste Android aplikacije [22]. Arhitektura Android operacijskog sustava prikazana je na slici 1.

Linux jezgra predstavlja temelj sustava i zadužena je za upravljanje hardverskim resursima uređaja, kao što su procesor, memorija, mrežna komunikacija i upravljački programi. U razvijenoj aplikaciji ovaj sloj omogućuje pristup hardverskim komponentama uređaja, primjerice kameri koja se koristi za skeniranje QR kodova.

Sloj biblioteka pruža dodatne funkcionalnosti potrebne aplikacijama, poput rada s grafikom, bazama podataka, mrežom i vanjskim servisima. U ovom radu korištene su biblioteke Firebase SDK, Google ML Kit i AndroidX.

Android Runtime (ART) predstavlja okruženje u kojem se izvršava programski kod Android aplikacija. Budući da je aplikacija razvijena u programskom jeziku Java, njezin se programski kod izvršava unutar tog okruženja.

Aplikacijski okvir pruža programska sučelja za upravljanje aktivnostima, korisničkim sučeljem, resursima i ostalim funkcionalnostima Android sustava. Razvijena aplikacija koristi ovaj sloj kroz aktivnosti poput LoginActivity i QrScannerActivity te ostale zaslone aplikacije.

Na vrhu arhitekture nalazi se aplikacijski sloj, kojem pripadaju korisničke aplikacije. Mobilna aplikacija razvijena u ovom radu pripada tom sloju te korisniku omogućuje prijavu, skeniranje QR koda, pregled jelovnika, komunikaciju s AI asistentom i izradu narudžbe.

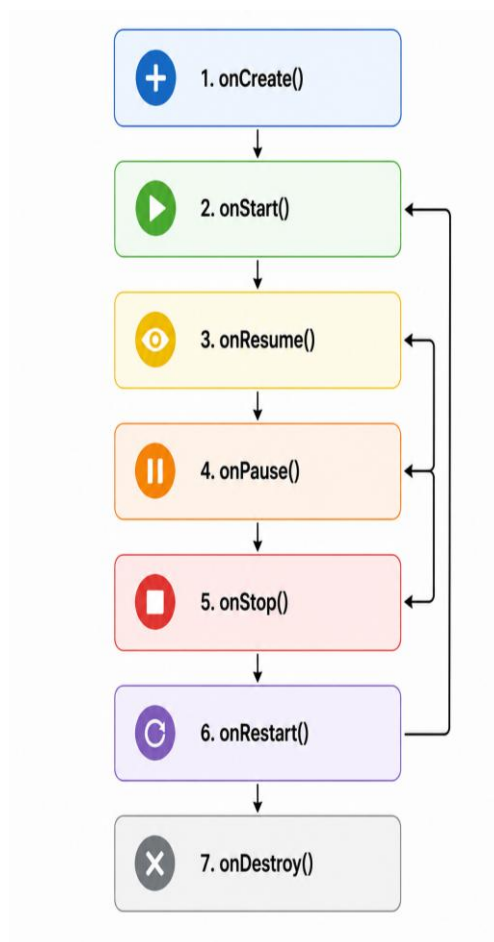
Opisana višeslojna arhitektura Android operacijskog sustava omogućila je razvoj aplikacije uz jasno razdvajanje odgovornosti pojedinih slojeva. U razvijenom sustavu aplikacijski sloj obuhvaća korisničko sučelje i poslovnu logiku aplikacije, aplikacijski okvir omogućuje upravljanje aktivnostima i pristup funkcionalnostima operacijskog sustava, biblioteke pružaju podršku za Firebase SDK, Google ML Kit i ostale korištene komponente, Android Runtime okruženje izvršava programski kod napisan u Javi, dok Linux jezgra upravlja hardverskim resursima uređaja, uključujući kameru koja se koristi za skeniranje QR kodova.



Slika 1. Arhitektura Android operacijskog sustava. Izvor: izrada autora prema Android Developers [22].

2.9 Životni ciklus Android aktivnosti

Životni ciklus Android aktivnosti (engl. *Android Activity Lifecycle*) predstavlja skup stanja kroz koja aktivnost prolazi tijekom svog izvršavanja. Android operacijski sustav upravlja prijelazima između tih stanja ovisno o korisnikovim radnjama i stanju uređaja. Razumijevanje životnog ciklusa aktivnosti važno je za razvoj stabilnih i učinkovitih aplikacija jer omogućuje pravilno upravljanje resursima, očuvanje podataka te prilagodbu rada aplikacije tijekom njezina korištenja. Životni ciklus Android aktivnosti prikazan je na slici 2 [23].



Slika 2. Životni ciklus Android aktivnosti. Izvor: Android Developers [23].

Android aktivnost tijekom izvođenja prolazi kroz nekoliko osnovnih metoda životnog ciklusa:

- `onCreate()` – poziva se prilikom stvaranja aktivnosti te služi za inicijalizaciju korisničkog sučelja i ostalih potrebnih komponenti.
- `onStart()` – aktivnost postaje vidljiva korisniku.

- `onResume()` – aktivnost prelazi u aktivno stanje te korisnik može komunicirati s njom.
- `onPause()` – poziva se kada aktivnost više nije u prvom planu.
- `onStop()` – aktivnost više nije vidljiva korisniku.
- `onRestart()` – poziva se prilikom ponovnog pokretanja prethodno zaustavljene aktivnosti.
- `onDestroy()` – poziva se prije konačnog uništavanja aktivnosti radi oslobađanja zauzetih resursa [23].

U razvijenoj aplikaciji metode životnog ciklusa koriste se za inicijalizaciju korisničkog sučelja, povezivanje s Firebase servisima, upravljanje korisničkom autentifikacijom te pokretanje pojedinih funkcionalnosti aplikacije. Njihovom primjenom osigurava se pravilno upravljanje stanjem aktivnosti i ponašanjem aplikacije tijekom korisničke interakcije.

3 Implementacija mobilne aplikacije

Mobilna aplikacija predstavlja korisnički dio razvijenog sustava i implementirana je za Android platformu korištenjem programskog jezika Java. Aplikacija omogućuje anonimnu prijavu korisnika, identifikaciju stola skeniranjem QR koda, pregled i prilagodbu digitalne ponude, izradu narudžbe, komunikaciju s AI asistentom te praćenje statusa narudžbe.

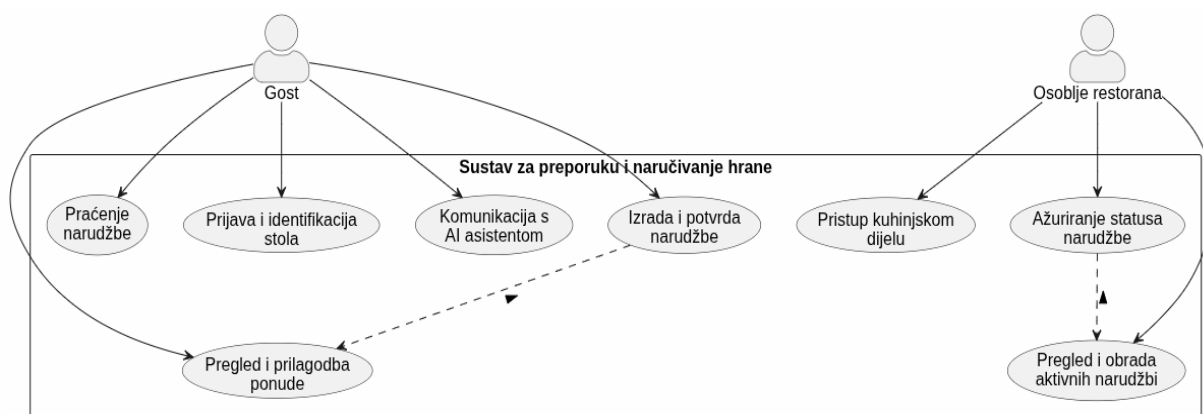
Podaci potrebni za rad aplikacije dohvaćaju se i pohranjuju korištenjem usluga Firebase platforme. Firebase Realtime Database koristi se za pohranu i dohvat podataka o stolovima, jelovniku i narudžbama, dok se komunikacija s poslužiteljskim dijelom AI asistenta ostvaruje putem Firebase Functions. Implementacija poslužiteljskog dijela sustava detaljnije je opisana u poglavlju 4.

U ovom poglavlju prikazani su struktura i organizacija mobilne aplikacije te implementacija njezinih glavnih funkcionalnosti, od prijave korisnika i identifikacije stola do sastavljanja, slanja i praćenja narudžbe.

3.1 Dijagram slučajeva korištenja

Radi prikaza osnovnih funkcionalnosti razvijenog sustava i korisnika koji s njima komuniciraju izrađen je UML dijagram slučajeva korištenja. U sustavu se razlikuju dvije osnovne skupine korisnika: gost ugostiteljskog objekta i osoblje restorana. Gost koristi funkcionalnosti povezane s identifikacijom stola, pregledom i prilagodbom ponude, komunikacijom s AI asistentom, izradom narudžbe i praćenjem njezina statusa. Osoblju restorana namijenjene su funkcionalnosti povezane s pregledom pristiglih narudžbi i upravljanjem njihovim statusima.

Na slici 3 prikazan je UML dijagram slučajeva korištenja razvijenog sustava, iz kojeg je vidljiva podjela funkcionalnosti između gosta i osoblja restorana.



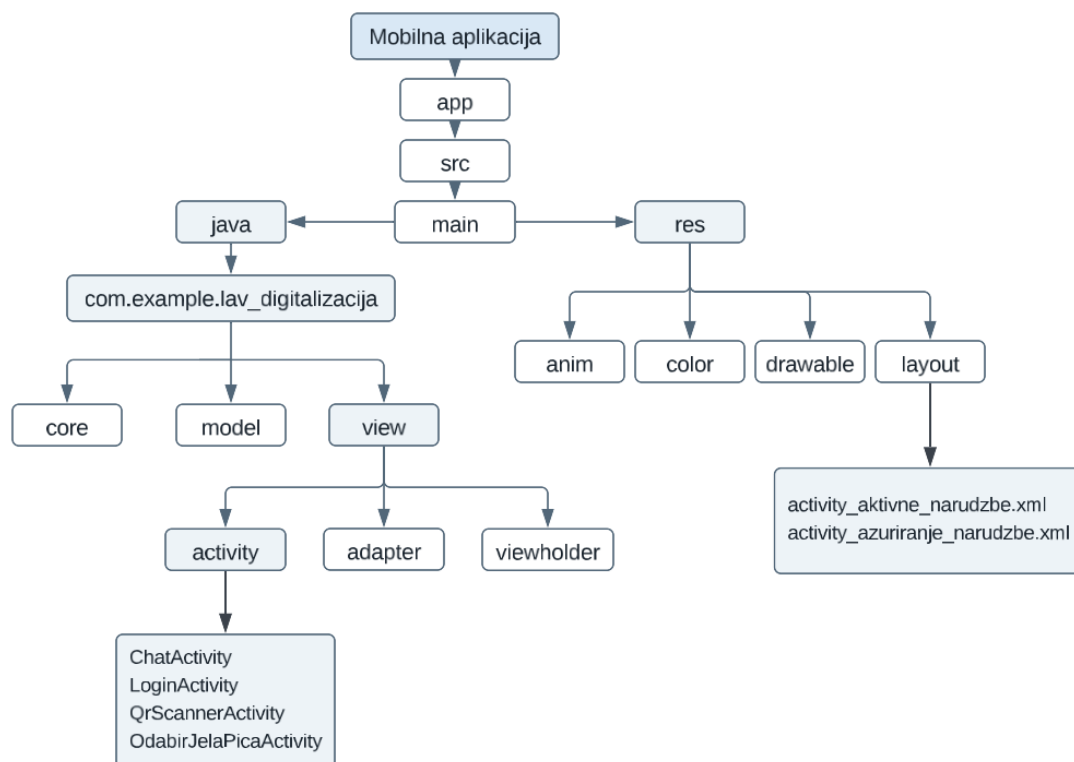
Slika 3. UML dijagram slučajeva korištenja razvijenog sustava. Izvor: izrada autora.

Gost nakon prijave i identifikacije stola može pregledavati i prilagođavati ponudu, komunicirati s AI asistentom te izraditi i potvrditi narudžbu. Komunikacija s AI asistentom povezana je s pregledom i prilagodbom ponude jer asistent korisniku može pomoći pri odabiru proizvoda i sastavljanju narudžbe. Nakon potvrde narudžbe gost može pratiti njezino stanje tijekom obrade.

Osoblje restorana pristupa kuhinjskom dijelu sustava, u kojem pregledava i obrađuje aktivne narudžbe te ažurira njihov status tijekom pripreme i posluživanja. Promjene statusa narudžbe pohranjuju se u Firebase Realtime Database te se automatski prikazuju gostu koji prati svoju narudžbu. Na taj način dijagram pokazuje podjelu odgovornosti između gosta i osoblja restorana. Gost sudjeluje u procesu od odabira ponude do praćenja narudžbe, dok je osoblje restorana zaduženo za obradu zaprimljenih narudžbi i ažuriranje njihova statusa.

3.2 Arhitektura i organizacija mobilne aplikacije

Mobilna aplikacija razvijena je u razvojnom okruženju Android Studio korištenjem programskog jezika Java. Projekt slijedi uobičajenu strukturu Android aplikacije, uz organizaciju izvornog koda prema funkcionalnim cjelinama. Središnji dio aplikacije nalazi se u direktoriju app/src/main, unutar kojeg su smješteni izvorni kod i resursi potrebni za rad aplikacije. Pojednostavljena struktura mobilne aplikacije prikazana je na slici 4.



Slika 4. Struktura mobilne aplikacije. Izvor: izrada autora.

Kao što je prikazano na slici 4, direktorij main podijeljen je na dio koji sadrži izvorni kod aplikacije i direktorij s resursima. Unutar Java paketa `com.example.lav_digitalizacija` izvorni kod organiziran je u direktorije `core`, `model` i `view`. Direktorij `core` sadrži zajedničke pomoćne funkcionalnosti, dok se u direktoriju `model` nalaze klase koje predstavljaju podatke kojima aplikacija upravlja. Direktorij `view` obuhvaća aktivnosti, adaptere i `ViewHolder` klase povezane s prikazom i obradom podataka u korisničkom sučelju.

Direktorij `res` sadrži resurse potrebne za prikaz korisničkog sučelja. XML datoteke koje definiraju raspored elemenata pojedinih zaslona nalaze se u direktoriju `layout`, dok su ostali resursi, poput grafičkih elemenata, boja i animacija, organizirani u direktorijima `drawable`, `color` i `anim`.

Korisničko sučelje aplikacije organizirano je korištenjem Android aktivnosti, pri čemu svaka aktivnost predstavlja zaseban funkcionalni zaslon. Prijelazi između aktivnosti ostvaruju se mehanizmom `Intent`, kojim se prema potrebi proslijeđuju podaci potrebni za nastavak procesa naručivanja.

Početnu točku aplikacije predstavlja `LoginActivity`, nakon koje korisnik prelazi na `QrScannerActivity` radi identifikacije stola. Nakon uspješne provjere QR koda otvara se `OdabirJelaPicaActivity`, iz kojeg korisnik može pregledavati ponudu, prilagođavati proizvode,

pristupiti AI asistentu i upravljati narudžbom. Ostale aktivnosti omogućuju pregled i potvrdu narudžbe, praćenje njezina statusa te pristup kuhinjskom dijelu aplikacije namijenjenom osoblju restorana.

Ovakva organizacija odvaja podatkovne modele, korisničko sučelje, pomoćne funkcionalnosti i resurse aplikacije te olakšava pregled i održavanje izvornog koda. Razdvajanje aplikacije na funkcionalne cjeline i komponente s jasno definiranim odgovornostima jedan je od pristupa kojim se može poboljšati održivost i testabilnost Android aplikacija [24]. Za organizaciju korisničkog sučelja alternativno su se mogli koristiti fragmenti ili deklarativni pristup Jetpack Compose. U razvijenom sustavu aktivnosti su zadržane zbog jasno odvojenih funkcionalnih zaslona i postojeće strukture projekta.

3.3 Početna prijava korisnika

Početni zaslon implementiran je u aktivnosti LoginActivity. Nakon pokretanja aplikacije automatski se izvršava anonimna autentifikacija putem usluge Firebase Authentication, zbog čega korisnik ne mora izrađivati korisnički račun ni unositi podatke za prijavu.

Takav pristup odabran je jer je aplikacija namijenjena gostima ugostiteljskog objekta koji nakon dolaska za stol žele odmah započeti proces naručivanja. Uvođenje obvezne registracije dodatno bi produžilo proces prije pristupa osnovnim funkcionalnostima aplikacije. Početni zaslon aplikacije prikazan je na slici 5.



Slika 5. Početni zaslon aplikacije. Izvor: izrada autora.

Nakon uspješne autentifikacije Firebase svakom anonimnom korisniku automatski dodjeljuje jedinstveni identifikator (UID), a aplikacija korisniku prikazuje dijaloški prozor za unos nadimka. Prije spremanja provjerava se je li nadimak unesen, a njegova je duljina ograničena na 15 znakova. Nadimak se koristi za prikaz informacija i prirodniju komunikaciju s AI asistentom.

Uneseni nadimak pohranjuje se lokalno korištenjem mehanizma SharedPreferences, koji je prikladan za spremanje manjih količina jednostavnih podataka u obliku parova ključ–vrijednost. Time nadimak ostaje dostupan i nakon ponovnog pokretanja aplikacije. Alternativno bi se mogla koristiti lokalna baza podataka Room, ali bi za pohranu nadimka i osnovnih postavki takvo rješenje nepotrebno povećalo složenost aplikacije. Ostali podaci, poput jelovnika, narudžbi i povijesti razgovora, pohranjuju se u Firebase Realtime Database.

3.4 Skeniranje QR koda

Nakon anonimne prijave korisnik prelazi na aktivnost QrScannerActivity, zaduženu za skeniranje QR koda postavljenog na stolu ugostiteljskog objekta. QR kod služi za identifikaciju restorana i stola s kojeg se narudžba šalje.

Za skeniranje se koristi Google Code Scanner API, konfiguriran tako da prepoznaje isključivo QR kodove. Nakon skeniranja aplikacija dohvaća tekstualnu vrijednost koda i provjerava njezin format.

U razvijenom sustavu QR kod mora započinjati oznakom LAVQR:, nakon koje slijedi jedinstveni token stola. Ako format nije ispravan ili token nedostaje, korisniku se prikazuje poruka o pogrešci i omogućuje ponovno skeniranje.

Nakon provjere formata aplikacija u bazi Firebase Realtime Database provjerava postoji li zapis povezan s izdvojenim tokenom te je li pripadajući stol aktivan. Podaci o stolovima pohranjeni su unutar čvora stolovi, pri čemu token služi kao ključ za dohvat odgovarajućeg zapisa. Postupak provjere tokena i dohvaćanja podataka o stolu prikazan je u isječku programskog koda 1.

Isječak programskog koda 1. Provjera tokena stola u Firebase Realtime Database

```
DatabaseReference stolRef = FirebaseDatabase.getInstance()
    .getReference("stolovi")
    .child(token);

stolRef.addListenerForSingleValueEvent(new ValueEventListener() {
    @Override
    public void onDataChange(@NonNull DataSnapshot snapshot) {
        if (!snapshot.exists()) {
            prikaziNeispravanQr();
            return;
        }

        Boolean aktivan = snapshot.child("aktivan").getValue(Boolean.class);
        Integer brojStola = snapshot.child("brojStola").getValue(Integer.class);
        String restoran = snapshot.child("restoran").getValue(String.class);

        if (aktivan == null || !aktivan || brojStola == null || restoran == null) {
            prikaziNeispravanQr();
            return;
        }

        prikaziUspjesanQr(brojStola, restoran, token);
    }
});
```

Kao što je prikazano u isječku programskog koda 1, nakon pronalaska zapisa provjerava se je li stol aktivan te postoje li podaci o broju stola i restoranu. Ako zapis ne postoji, stol nije aktivan ili nedostaje neki od potrebnih podataka, QR kod smatra se neispravnim. Ako je provjera uspješna, korisniku se prikazuje broj stola i otvara se aktivnost OdabirJelaPicaActivity. U nju se proslijeđuju broj stola, naziv restorana i QR token kako bi se buduća narudžba povezala s odgovarajućim stolom.

Alternativno rješenje bilo bi ručno unošenje ili odabir broja stola s unaprijed definirane liste. Takav pristup ne bi zahtijevao uporabu kamere, ali bi povećao mogućnost pogrešnog odabira i zahtijevao dodatno održavanje podataka o stolovima. Primjena QR koda zato predstavlja jednostavnije i pouzdanije rješenje za identifikaciju mjesta naručivanja.

3.5 Prikaz digitalnog jelovnika

Nakon uspješne validacije QR koda korisnik prelazi na aktivnost OdabirJelaPicaActivity, koja predstavlja glavni zaslon za pregled ponude hrane i pića. Korisnik može pregledavati proizvode prema kategorijama, otvoriti pojedini proizvod, pristupiti AI asistentu i pregledati trenutačnu narudžbu.

Pri otvaranju aktivnosti podaci o proizvodima dohvaćaju se iz Firebase Realtime Database, čime se korisniku prikazuje aktualna ponuda restorana. Proizvodi su organizirani prema kategorijama kako bi se korisniku olakšalo pregledavanje jelovnika i pronalaženje željenog proizvoda. Odabirom proizvoda otvara se aktivnost ModifikacijeActivity, u kojoj korisnik prije dodavanja u narudžbu može odabrati količinu i dostupne modifikacije te unijeti dodatnu napomenu.

Alternativno rješenje bila bi lokalna pohrana jelovnika na uređaju, koja bi smanjila broj mrežnih zahtjeva i omogućila pristup ponudi bez internetske veze. Međutim, takav pristup zahtijevao bi dodatnu sinkronizaciju cijena, dostupnosti proizvoda i ostalih podataka. Budući da se ponuda restorana može mijenjati, odabran je izravan dohvat podataka iz Firebase Realtime Database kako bi aplikacija prikazivala trenutačno dostupne podatke.

3.6 Prilagodba proizvoda i dodavanje u narudžbu

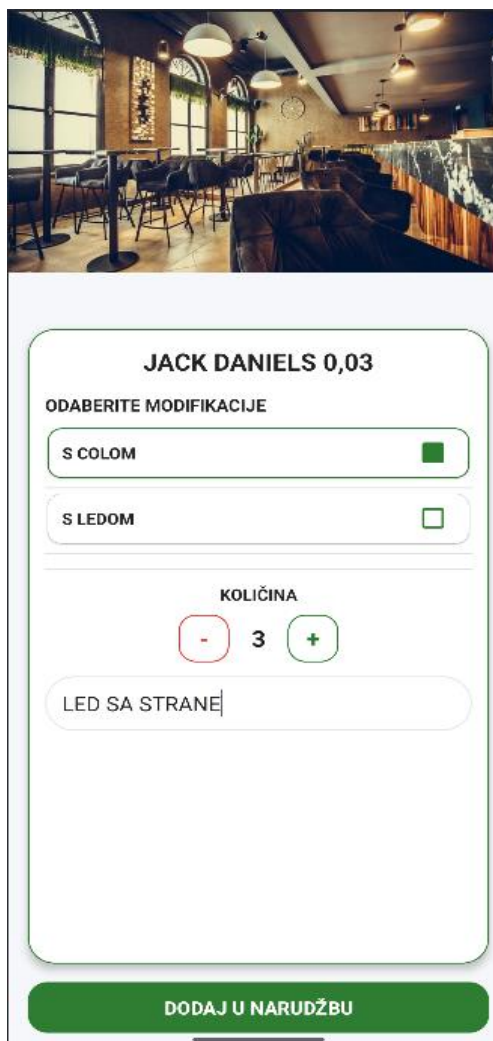
Nakon odabira proizvoda korisnik prelazi na aktivnost ModifikacijeActivity, u kojoj može odabrati dostupne modifikacije, promijeniti količinu i unijeti dodatnu napomenu. Na slici 6

prikazan je izgled aktivnosti i osnovne mogućnosti dostupne korisniku prilikom prilagodbe odabranog proizvoda.



Slika 6. Aktivnost za prilagodbu proizvoda. Izvor: izrada autora.

Aktivnost pri otvaranju prikazuje osnovne podatke o proizvodu i iz baze Firebase Realtime Database dohvaća dostupne modifikacije. Prikazane mogućnosti ovise o odabranom proizvodu te mogu uključivati okus, prilog, razinu ljutine ili druge opcije definirane u bazi. Na slici 7 prikazan je primjer odabira dostupnih modifikacija prije dodavanja proizvoda u narudžbu.



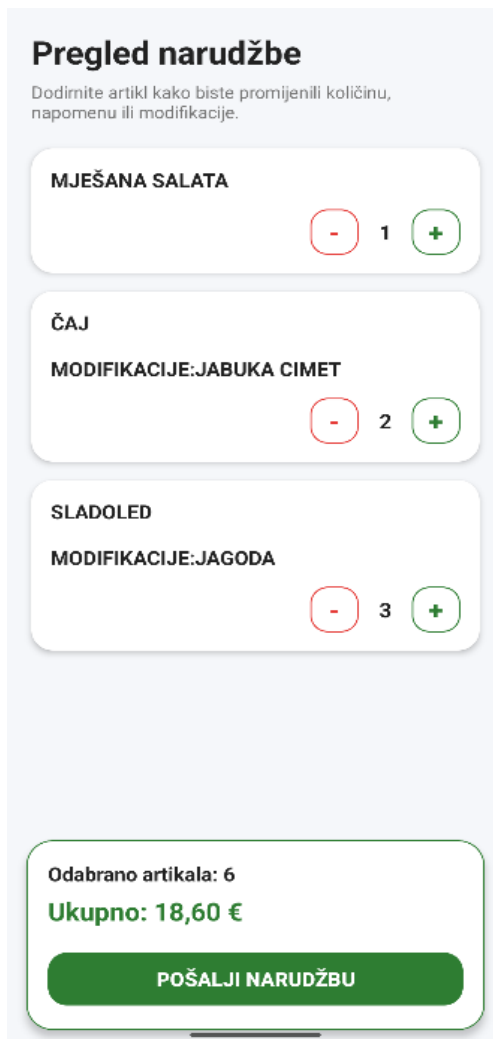
Slika 7. Prilagodba proizvoda prije dodavanja u narudžbu. Izvor: izrada autora.

Nakon potvrde aplikacija izrađuje stavku narudžbe koja sadrži podatke o proizvodu, količini, odabranim modifikacijama i korisničkoj napomeni. Stavka se privremeno pohranjuje lokalno korištenjem mehanizma SharedPreferences, čime se korisniku omogućuje postupno sastavljanje narudžbe prije njezina konačnog slanja.

Alternativno bi se svaka dodana ili izmijenjena stavka mogla odmah pohraniti u Firebase Realtime Database. Takav pristup omogućio bi sinkronizaciju podataka između više uređaja, ali bi povećao broj mrežnih zahtjeva i zahtijevao upravljanje nepotvrđenim narudžbama. Budući da narudžba postaje valjana tek nakon korisnikove potvrde, odabrana je lokalna pohrana tijekom njezina sastavljanja, dok se konačna narudžba u bazu šalje tek nakon potvrde korisnika.

3.7 Pregled narudžbe

Nakon što korisnik doda jedan ili više proizvoda u narudžbu, prelazi na aktivnost PregledNarudzbiActivity, u kojoj se prikazuju sve trenutačno odabrane stavke, njihove količine i ukupna cijena narudžbe. Aktivnost dohvaća podatke iz lokalne pohrane SharedPreferences i prikazuje ih s pomoću komponente RecyclerView. Na slici 8 prikazan je zaslon za pregled narudžbe prije njezina konačnog slanja.



Slika 8. Pregled narudžbe prije slanja. Izvor: izrada autora.

Korisnik na prikazanom zaslonu može pregledati odabrane proizvode, njihove količine i ukupnu cijenu narudžbe. Odabirom pojedine stavke ponovno se otvara aktivnost ModifikacijeActivity, u kojoj korisnik može promijeniti količinu, modifikacije ili napomenu. Proizvod je moguće ukloniti povlačenjem stavke ulijevo ili udesno, nakon čega se prikazuje mogućnost vraćanja uklonjene stavke kako bi se smanjila mogućnost slučajnog brisanja.

Nakon potvrde aplikacija provjerava sadrži li narudžba barem jednu stavku. Ako narudžba nije prazna, provodi se transakcija nad brojačem ukupno izdanih narudžbi. Time se osigurava da svaka narudžba dobije jedinstvenu oznaku i u slučaju istodobnog slanja više narudžbi. U bazu Firebase Realtime Database zatim se pohranjuju stavke narudžbe, vrijeme izrade, broj stola, restoran, QR token, identifikator korisnika i početni status. Postupak spremanja potvrđene narudžbe prikazan je u isječku programskog koda 2.

Isječak programskog koda 2. Slanje narudžbe u Firebase Realtime Database

```
Map<String, Object> narudzba = new HashMap<>();
narudzba.put("stavke", new ArrayList<>(narudzbeList));
narudzba.put("vrijeme", vrijeme);
narudzba.put("brojStola", tableNumber);
narudzba.put("restaurant", restaurant);
narudzba.put("qrToken", qrToken);
narudzba.put("userId", userId);
narudzba.put("status", "Narudžba zaprimljena");
narudzbeRef.child(noviKljuc).setValue(narudzba);
```

U isječku programskog koda 2 prikazan je pojednostavljeni postupak spremanja potvrđene narudžbe u Firebase Realtime Database. Nakon uspješnog spremanja lokalni podaci brišu se iz SharedPreferences, a korisnik se preusmjerava na zaslon potvrde narudžbe.

Narudžba se u bazu sprema tek nakon korisnikove konačne potvrde. Alternativno bi se svaka promjena mogla odmah sinkronizirati s bazom podataka, ali bi takav pristup povećao broj mrežnih zahtjeva i zahtijevao upravljanje nedovršenim ili napuštenim narudžbama. Odabrano rješenje omogućuje da se u bazu pohranjuju samo potvrđene narudžbe.

3.8 AI asistent u mobilnoj aplikaciji

AI asistent implementiran je u aktivnosti ChatActivity i omogućuje komunikaciju prirodnim jezikom tijekom procesa naručivanja. Korisnik može postavljati pitanja o jelovniku, zatražiti preporuku hrane i pića, sastaviti obrok, dodati proizvode u narudžbu ili ponoviti prethodnu narudžbu.

Za razliku od klasičnog digitalnog jelovnika, u kojem korisnik samostalno pregledava kategorije, razgovorni pristup olakšava pronalazak odgovarajućih proizvoda kada korisnik nema unaprijed određen izbor. Korisničko sučelje sastoji se od zaglavlja, popisa poruka prikazanih komponentom RecyclerView i polja za unos teksta. Na slici 9 prikazan je zaslon AI asistenta tijekom komunikacije s korisnikom.



Slika 9. Zaslon AI asistenta. Izvor: izrada autora.

Aktivnost pri otvaranju dohvaća trenutačno prijavljenog korisnika. Njegov jedinstveni identifikator koristi se za pristup povijesti razgovora pohranjenoj u bazi Firebase Realtime Database. Iz čvora chat_history dohvaća se posljednjih dvadeset poruka, čime se korisniku omogućuje pregled prethodne komunikacije i nastavak razgovora u odgovarajućem kontekstu.

Nakon što korisnik pošalje poruku, ona se prikazuje u korisničkom sučelju i pohranjuje u povijest razgovora. Potom se metodom `sendToAI()` priprema zahtjev i putem usluge

Firebase Functions poziva funkcija `chatWaiter()`. Postupak slanja korisničkog zahtjeva poslužiteljskom dijelu prikazan je u isječku programskog koda 3.

Isječak programskog koda 3. Slanje korisničke poruke AI asistentu

```
private void sendToAI(String userText) {  
  
    Map<String, Object> data = buildBasePayload();  
  
    data.put("message", userText);  
    data.put("action", "");  
    data.put("mode", activeMode);  
  
    btnSend.setEnabled(false);  
  
    functions.getHttpsCallable("chatWaiter")  
        .call(data)  
        .addOnSuccessListener(result -> {  
  
            btnSend.setEnabled(true);  
  
            ParsedResponse parsed = parseResponse(result.getData());  
            applyResponse(parsed);  
  
        });  
}
```

Komunikacija s OpenAI API-jem ne odvija se izravno iz mobilne aplikacije, nego putem usluge Firebase Functions. Alternativno bi mobilna aplikacija mogla izravno pozivati OpenAI API, ali bi takvo rješenje zahtijevalo pohranu API ključa u klijentskom dijelu aplikacije, gdje bi ga bilo moguće izdvojiti i zloupotrijebiti. Posredovanjem funkcije `chatWaiter()` API ključ i poslovna logika ostaju na poslužiteljskoj strani.

Uz korisničku poruku poslužiteljskom dijelu prosljeđuju se podaci potrebni za očuvanje konteksta razgovora, uključujući identifikator i nadimak korisnika, podatke o stolu i restoranu, QR token te povijest razgovora. Slanjem dodatnog konteksta model može povezati novi zahtjev s prethodnim porukama te podacima relevantnima za trenutačnu sesiju. Slanje samo posljednje poruke smanjilo bi količinu podataka i potrošnju tokena, ali bi otežalo razumijevanje kratkih ili kontekstualnih zahtjeva.

Nakon obrade zahtjeva mobilna aplikacija prima strukturirani odgovor koji može sadržavati tekstualnu poruku, vrstu korisničke namjere, dodatne podatke i proizvode koje treba dodati u košaricu. Metoda `parseResponse()` pretvara primljene podatke u oblik prikladan za daljnju obradu, nakon čega se odgovor prikazuje u razgovoru i pohranjuje u povijest.

Ako odgovor sadrži stavke narudžbe, aplikacija ih može dodati u lokalnu košaricu. AI

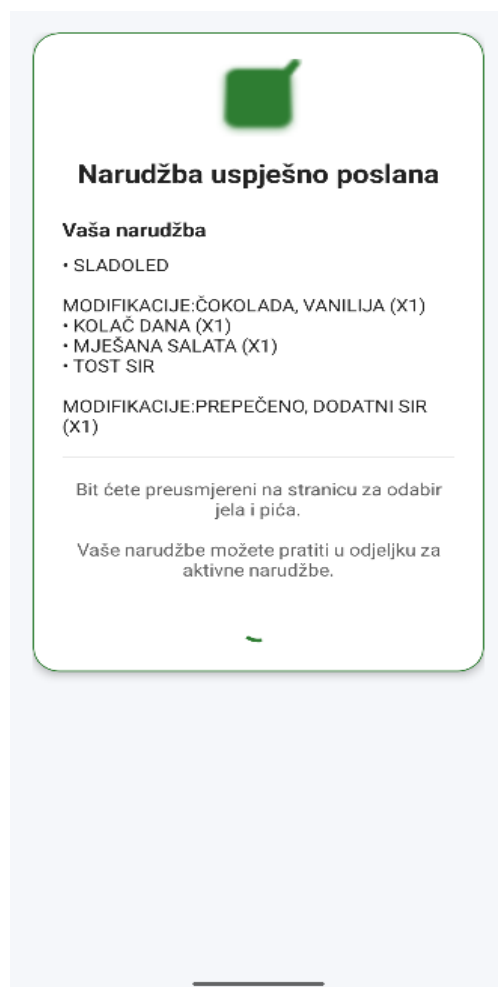
asistent tako ne služi samo za prikaz informacija, nego sudjeluje i u sastavljanju narudžbe. Korisnik pritom zadržava kontrolu jer prije konačnog slanja može pregledati, izmijeniti ili ukloniti dodane stavke.

Asistent može uzeti u obzir postojeći sadržaj košarice, predložiti povezane proizvode ili upozoriti korisnika da se određena stavka već nalazi u narudžbi. Kod zahtjeva za ponavljanjem prethodne narudžbe korisniku se prikazuje potvrda prije dodavanja proizvoda, čime se smanjuje mogućnost neželjenih promjena.

Razdvajanjem korisničkog sučelja u aktivnosti ChatActivity od obrade zahtjeva u funkciji chatWaiter() postignuta je sigurnija i modularnija arhitektura. Poslužiteljska implementacija funkcije chatWaiter(), obrada korisničke namjere i komunikacija s velikim jezičnim modelom detaljnije su opisane u poglavlju 4.

3.9 Potvrda narudžbe

Nakon što korisnik pošalje narudžbu, aplikacija ga preusmjerava na aktivnost PotvrdaNarudzbeActivity. Ova aktivnost služi za prikaz potvrde da je narudžba uspješno poslana u kuhinju. Korisniku se prikazuje poruka o uspješnom slanju narudžbe te popis poslanih stavki, čime odmah dobiva povratnu informaciju da je narudžba zaprimljena. Zaslون potvrde narudžbe prikazan je na slici 10.



Slika 10. Potvrda uspješno poslane narudžbe. Izvor: izrada autora.

Nakon prikaza potvrde korisnik se automatski vraća na zaslon za odabir jela i pića, a preusmjeravanje se izvršava nakon nekoliko sekundi. Automatski povratak na glavni zaslon odabran je kako bi korisnik mogao odmah nastaviti pregledavati jelovnik ili započeti novu narudžbu bez dodatnih koraka. Alternativno rješenje bilo bi zadržavanje korisnika na zaslonu potvrde do ručnog zatvaranja, no takav pristup zahtijevao bi dodatnu korisničku interakciju.

3.10 Pregled aktivnih narudžbi

Nakon uspješnog slanja narudžbe korisnik može otvoriti aktivnost AktivneNarudzbeActivity, u kojoj se prikazuju njegove narudžbe koje su još u postupku pripreme ili posluživanja. Time korisnik može provjeriti njihov trenutni status bez dodatnog obraćanja osoblju restorana. Na slici 11 prikazan je zaslon s aktivnim narudžbama korisnika i njihovim trenutnim statusima.



Slika 11. Pregled aktivnih narudžbi. Izvor: izrada autora.

Narudžbe se prikazuju s pomoću komponente RecyclerView. Tijekom dohvaćanja podataka prikazuje se indikator učitavanja, dok se u slučaju nepostojanja aktivnih narudžbi korisniku prikazuje odgovarajuća informativna poruka.

Pri pokretanju aktivnosti iz baze Firebase Realtime Database dohvaćaju se narudžbe

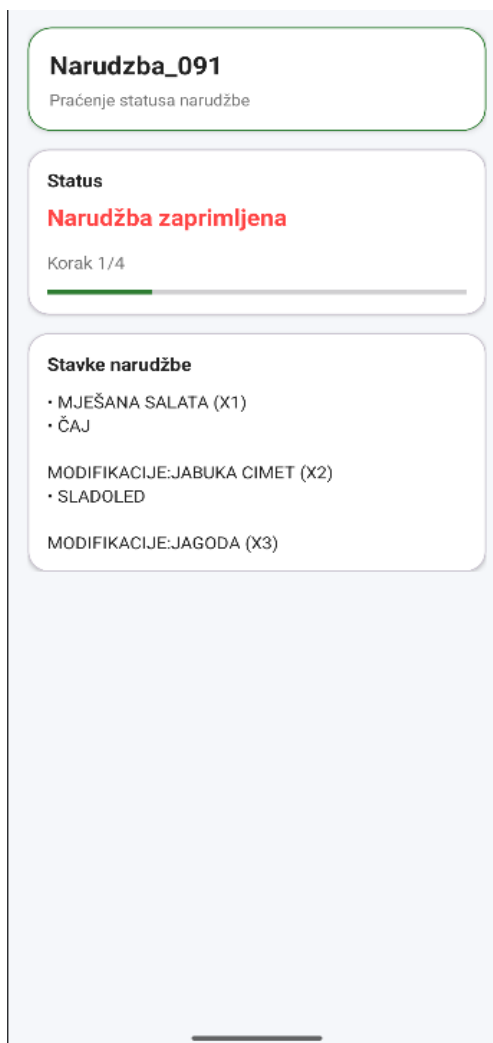
povezane s identifikatorom trenutno prijavljenog korisnika. Zatim se provjerava status svake narudžbe, a u prikaz se uključuju samo one koje nisu dovršene ni otkazane.

Prikaz samo aktivnih narudžbi odabran je kako bi korisničko sučelje ostalo usmjereno na trenutno relevantne informacije. Alternativno bi se mogle prikazivati i završene narudžbe, čime bi korisnik dobio povijest naručivanja, ali bi se povećala količina podataka i otežalo pronalaženje narudžbi koje su još u tijeku.

Odabirom narudžbe otvara se aktivnost `PracenjeNarudzbeActivity`, kojoj se proslijeđuju identifikator narudžbe te podaci o restoranu, stolu i QR tokenu. Aktivnost `AktivneNarudzbeActivity` tako predstavlja poveznicu između potvrde narudžbe i detaljnog praćenja njezina statusa.

3.11 Praćenje narudžbe

Nakon odabira aktivne narudžbe korisnik prelazi na aktivnost `PracenjeNarudzbeActivity`, koja omogućuje detaljan pregled narudžbe i praćenje njezina statusa od zaprimanja do posluživanja za stol. Na slici 12 prikazan je zaslon za praćenje narudžbe s trenutačnim statusom, indikatorom napretka i pripadajućim stavkama.



Slika 12. Praćenje statusa narudžbe. Izvor: izrada autora.

Na zaslonu se prikazuju osnovni podaci o narudžbi, trenutačni status, indikator napretka i popis naručenih stavki. Tekstualni status i vizualni indikator korisniku omogućuju jednostavno praćenje faze obrade bez potrebe za dodatnom komunikacijom s osobljem restorana.

Aktivnost pri otvaranju dohvaća identifikator odabrane narudžbe proslijeđen iz prethodnog zaslona. Na temelju njega pristupa se odgovarajućem zapisu u bazi Firebase Realtime Database i registrira oslušivač promjena. Način praćenja promjene statusa

prikazan je u isječku programskog koda 4.

Isječak programskog koda 4. Praćenje promjene statusa narudžbe

```
narudzbaref.addValueEventListener(new ValueEventListener()
{
    @Override
    public void onDataChange(@NonNull DataSnapshot snapshot) {

        String status = snapshot.child("status").getValue(String.class);
        Integer korak = snapshot.child("korak").getValue(Integer.class);
        Integer ukupnoKoraka = snapshot.child("ukupnoKoraka").getValue(Integer.class);

        txtStatus.setText(status);
        txtKorak.setText("Korak " + korak + "/" + ukupnoKoraka);

        progressBarNarudzba.setMax(ukupnoKoraka);
        progressBarNarudzba.setProgress(korak);
        // Dohvat stavki narudžbe...
    }

    @Override
    public void onCancelled(@NonNull DatabaseError error) {
    }
});
```

U isječku programskog koda 4 prikazana je registracija osluškivača nad zapisom narudžbe. Pri svakoj promjeni podataka automatski se ažuriraju status i indikator napretka, pa korisnik vidi trenutačno stanje narudžbe bez ručnog osvježavanja zaslona.

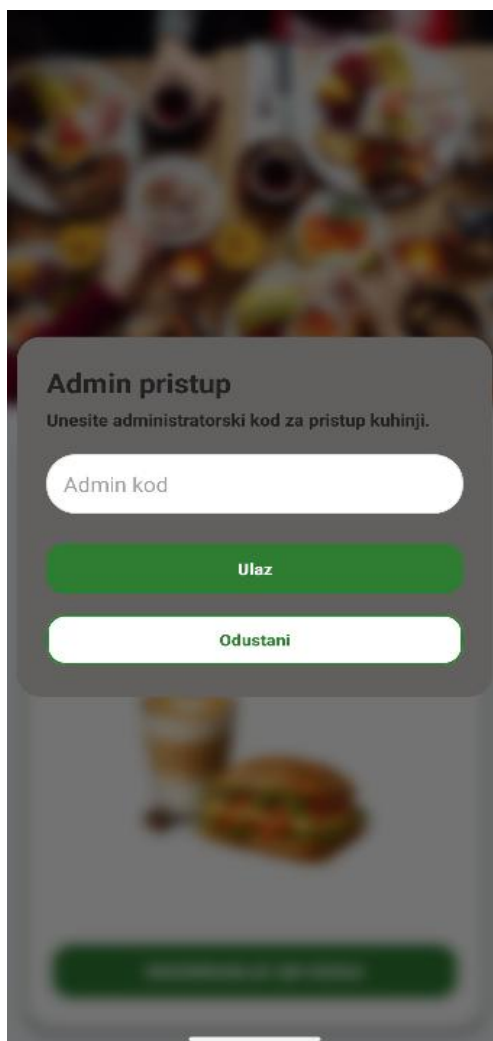
Alternativno rješenje bilo bi periodično dohvaćanje podataka u određenim vremenskim razmacima. Takav bi pristup mogao uzrokovati kašnjenje u prikazu promjena i povećati broj mrežnih zahtjeva. Primjenom sinkronizacije u stvarnom vremenu podaci se ažuriraju odmah nakon promjene statusa, uz izbjegavanje nepotrebnih upita prema poslužiteljskom dijelu sustava.

Status narudžbe prolazi kroz uzastopne faze pripreme i posluživanja. Promjene izvršava osoblje u kuhinjskom dijelu aplikacije, a Firebase Realtime Database automatski ih prosljeđuje korisničkom dijelu. Time se omogućuje pravodobno i transparentno praćenje narudžbe tijekom cijelog procesa.

3.12 Pristup kuhinjskom dijelu aplikacije

Osim dijela namijenjenog gostima, aplikacija sadrži i kuhinjski dio za osoblje. Osoblje kuhinjskom dijelu pristupa iz aktivnosti LoginActivity unosom administratorskog pristupnog koda u dijaloški prozor. Aplikacija provjerava je li vrijednost unesena te, u slučaju ispravnog

koda, otvara aktivnost KuhinjaActivity. Kod pogrešnog unosa prikazuje se odgovarajuća poruka. Na slici 13 prikazan je dijaloški prozor za unos administratorskog pristupnog koda.



Slika 13. Dijaloški prozor za pristup kuhinjskom dijelu aplikacije. Izvor: izrada autora.

Jednostavan pristupni kod odabran je jer je aplikacija razvijena kao prototip. Takvo rješenje nije prikladno za produkcijski sustav jer ne omogućuje pouzdanu identifikaciju osoblja ni upravljanje ovlastima. Sigurnije rješenje uključivalo bi zasebne korisničke račune u usluzi Firebase Authentication i dodjelu uloga, primjerice administratora, kuhara i konobara.

Nakon uspješne provjere otvara se aktivnost KuhinjaActivity, koja prikazuje aktivne narudžbe svih stolova. Za razliku od korisničkog dijela, u kojem korisnik vidi samo vlastite narudžbe, kuhinjski dio omogućuje osoblju pregled svih narudžbi koje zahtijevaju obradu. Na slici 14 prikazan je zaslon kuhinjskog dijela s pregledom aktivnih narudžbi.



Slika 14. Pregled aktivnih narudžbi u kuhinjskom dijelu aplikacije. Izvor: izrada autora.

Narudžbe se prikazuju s pomoću komponente RecyclerView, dok se tijekom njihova dohvaćanja prikazuje indikator učitavanja. Ako nema aktivnih narudžbi, osoblju se prikazuje odgovarajuća informativna poruka.

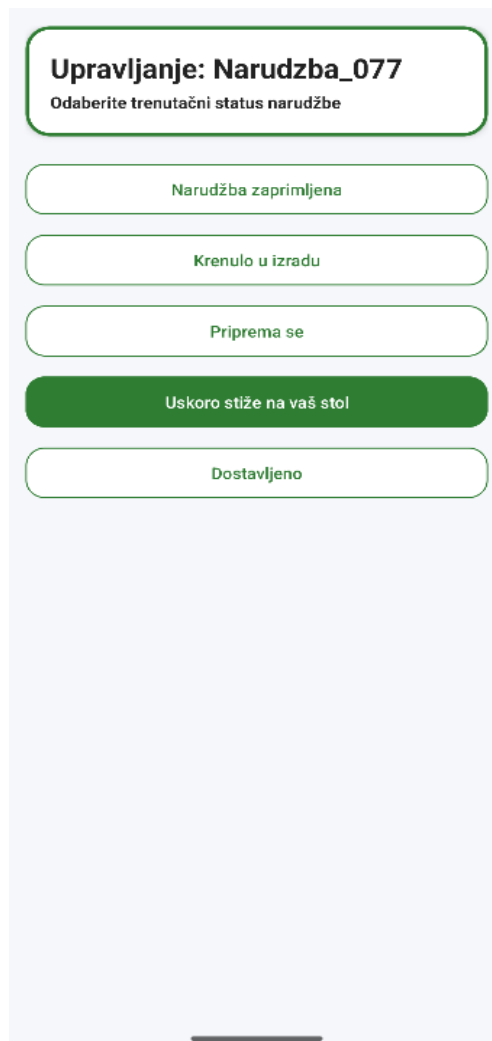
Podaci o narudžbama dohvaćaju se iz baze Firebase Realtime Database korištenjem osluškivača promjena. Nove narudžbe i promjene postojećih zapisa zato se prikazuju bez ručnog osvježavanja zaslona. Alternativno bi se podaci mogli periodično dohvaćati, ali bi takav pristup povećao broj mrežnih zahtjeva i mogao uzrokovati kašnjenje u prikazu novih narudžbi.

Aplikacija prikazuje samo narudžbe koje nemaju status „Dostavljeno” ili „Otkazano“. Time sučelje ostaje usmjereno na narudžbe koje još zahtijevaju postupanje osoblja. Prikaz završenih narudžbi mogao bi se naknadno implementirati kao zasebna povijest narudžbi.

3.13 Ažuriranje statusa narudžbe

Odabirom aktivne narudžbe u aktivnosti KuhinjaActivity otvara se aktivnost AzuriranjeNarudzbeActivity, u kojoj osoblje može mijenjati status narudžbe tijekom pripreme i posluživanja. Promjene statusa automatski se prikazuju korisniku u aktivnosti PracenjeNarudzbeActivity.

Pri otvaranju aktivnosti dohvaća se identifikator odabrane narudžbe i na temelju njega pristupa odgovarajućem zapisu u bazi Firebase Realtime Database. Ako identifikator nije dostupan, prikazuje se poruka o pogrešci i aktivnost se zatvara. Na slici 15 prikazan je zaslon na kojem osoblje može pregledati trenutni status i odabrati sljedeću fazu obrade narudžbe.



Slika 15. Aktivnost za ažuriranje statusa narudžbe. Izvor: izrada autora.

Pri pokretanju aktivnosti iz baze se dohvaća trenutni status te se označava odgovarajući gumb. Odabirom novog statusa ažuriraju se tekstualna oznaka statusa, redni broj trenutnog koraka i ukupan broj koraka u odgovarajućem zapisu narudžbe u bazi Firebase Realtime Database.

Tijekom spremanja gumbi se privremeno onemogućuju kako bi se spriječilo višestruko slanje istog zahtjeva. Bez takve zaštite osoblje bi pri sporijoj mrežnoj vezi moglo više puta pokrenuti promjenu statusa i time izazvati nepotrebna ažuriranja.

Nakon uspješnog spremanja osoblju se prikazuje potvrda, a aplikacija se vraća na pregled aktivnih narudžbi. Promjena statusa automatski se prikazuje i u korisničkom dijelu aplikacije jer `PracenjeNarudzbeActivity` koristi osluškivač promjena nad istim zapisom u bazi. Time korisnik dobiva ažurirani status narudžbe bez potrebe za ručnim osvježavanjem podataka.

4 Implementacija poslužiteljskog dijela

Poslužiteljski dio sustava implementiran je s pomoću usluge Firebase Functions, korištenjem okruženja Node.js i programskog jezika JavaScript. Firebase Functions izvršava poslužiteljsku logiku u oblaku prema modelu serverless, na zahtjev mobilne aplikacije, bez potrebe za postavljanjem i održavanjem vlastitog poslužitelja. Serverless pristup omogućuje izvršavanje aplikacijske logike bez izravnog upravljanja poslužiteljskom infrastrukturom, pri čemu pružatelj usluge preuzima upravljanje resursima i njihovim skaliranjem [14].

Firebase Functions odabran je zbog jednostavne integracije s uslugama Firebase Authentication i Firebase Realtime Database. Alternativno rješenje bio bi zaseban poslužiteljski sustav, primjerice razvijen s pomoću okruženja Node.js i biblioteke Express, koji bi izlagao REST API krajnje točke. Takav bi pristup pružio veću kontrolu nad infrastrukturom i konfiguracijom, ali bi zahtijevao dodatno održavanje, administraciju i upravljanje skaliranjem. Za opseg razvijenog sustava odabrani pristup omogućio je jednostavniju implementaciju i smanjio potrebu za upravljanjem infrastrukturom.

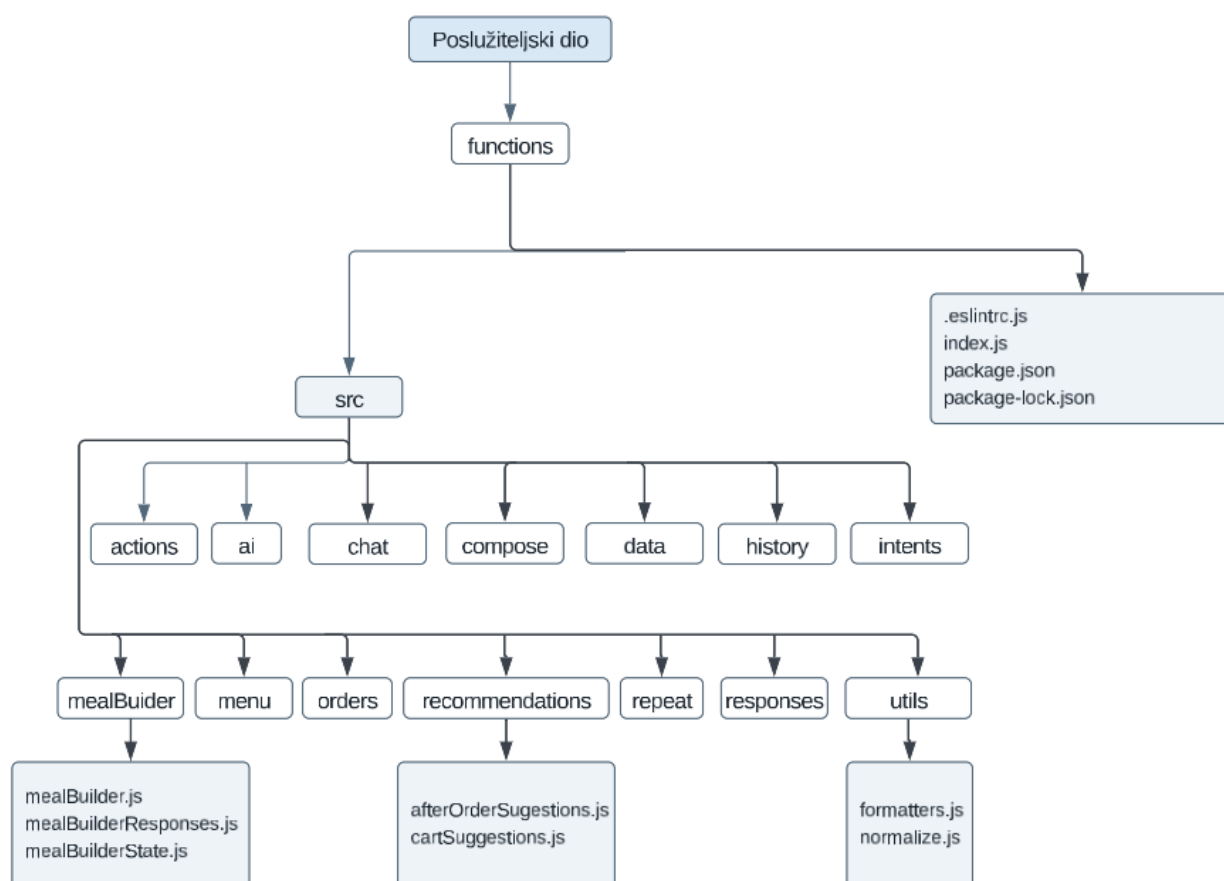
Glavna ulazna točka poslužiteljskog dijela nalazi se u datoteci `index.js`, u kojoj je definirana pozivna funkcija `chatWaiter()`. Funkcija prima zahtjeve iz aktivnosti `ChatActivity`, dohvaća potreban kontekst iz baze Firebase Realtime Database, obrađuje korisničku poruku i mobilnoj aplikaciji vraća strukturirani odgovor. Pritom povezuje module zadužene za prepoznavanje korisničke namjere, pretraživanje jelovnika, generiranje preporuka, obradu košarice i dohvat prethodnih narudžbi.

Za pristup podacima koristi se Firebase Admin SDK, koji poslužiteljskim funkcijama omogućuje rad s podacima o jelovniku, modifikacijama, korisnicima i narudžbama. Komunikacija s OpenAI API-jem odvija se isključivo na poslužiteljskoj strani, dok se API ključevi pohranjuju kao Firebase tajne. Time osjetljivi podaci nisu uključeni u mobilnu aplikaciju niti su dostupni krajnjem korisniku.

Poslužiteljski dio organiziran je u funkcionalne module s jasno razdvojenim odgovornostima. U nastavku poglavlja opisana je organizacija poslužiteljske logike, komunikacija između glavnih komponenti sustava te implementacija funkcionalnosti povezanih s AI asistentom.

4.1 Organizacija poslužiteljske logike

Radi jednostavnijeg održavanja i proširivanja, poslužiteljska logika podijeljena je u funkcionalne module unutar direktorija `src`, dok datoteka `index.js` predstavlja glavnu ulaznu točku sustava. Organizacija poslužiteljskog dijela prikazana je na slici 16.



Slika 16. Organizacija poslužiteljskog dijela projekta. Izvor: izrada autora.

Direktorij *actions* sadrži obradu akcija koje proizlaze iz komunikacije s AI asistentom, poput prikaza jelovnika, dohvaćanja popularnih proizvoda, ponavljanja prethodne narudžbe i dodavanja stavki u košaricu.

U direktoriju *ai* nalaze se moduli za komunikaciju s OpenAI API-jem, prepoznavanje korisničke namjere, planiranje tijeka razgovora i parsiranje narudžbi napisanih prirodnim jezikom. Direktorij *chat* zatim koristi rezultate obrade kako bi pripremio stavke narudžbe i odgovarajući odgovor korisniku.

Direktorij *mealBuilder* implementira postupno sastavljanje kompletnog obroka. Sustav pamti trenutačnu fazu razgovora i prethodne odabire te korisnika vodi kroz pojedine

kategorije proizvoda. Alternativno bi se cijela ponuda mogla prikazati odjednom, ali bi takav pristup kod većih jelovnika povećao količinu informacija koje korisnik mora pregledati.

Direktorij *menu* obuhvaća dohvat i pretraživanje jelovnika, prepoznavanje kategorija i alergena, povezivanje različitih naziva istog proizvoda te pronalaženje artikala prema korisničkom opisu. Direktorij *orders* zadužen je za obradu košarice, povijesti narudžbi i statusa aktivnih narudžbi.

Funkcije za generiranje prijedloga nalaze se u direktoriju *recommendations*. Preporuke se temelje na trenutačnom sadržaju košarice, prethodnim narudžbama, popularnosti proizvoda i pravilima za kombiniranje jela. Kombiniranjem više izvora podataka smanjuje se ovisnost o korisničkoj povijesti, koja kod novih korisnika može biti nedostupna.

Direktoriji *history*, *repeat* i *compose* sadrže pomoćne funkcionalnosti za rad s prethodnim razgovorima i narudžbama, ponavljanje ranijih odabira te oblikovanje personaliziranih prijedloga. Direktorij *responses* zadužen je za izradu strukturiranog odgovora koji može sadržavati tekstualnu poruku, vrstu odgovora, stavke narudžbe, dostupne akcije i dodatne metapodatke.

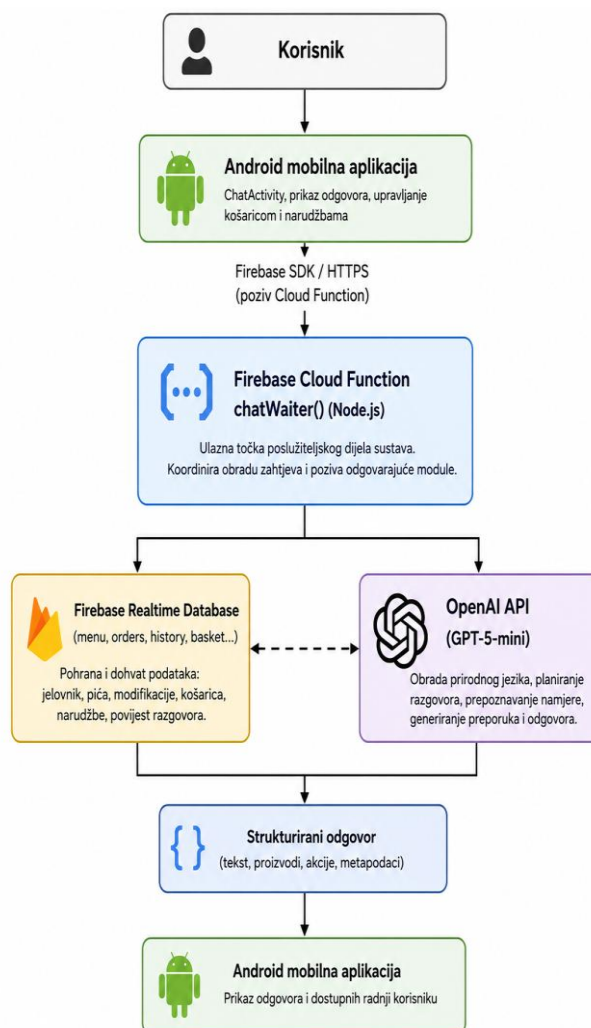
Zajedničke funkcije, poput normalizacije teksta, uklanjanja dijakritičkih znakova i ispravljanja čestih tipografskih pogrešaka, izdvojene su u direktorij *utils*.

Datoteka *index.js* koordinira rad navedenih modula. Nakon primitka zahtjeva dohvaća potreban kontekst iz baze Firebase Realtime Database, određuje korisničku namjeru, poziva odgovarajuće module i vraća strukturirani rezultat mobilnoj aplikaciji.

Alternativno bi se poslovna logika mogla implementirati u jednoj ili nekoliko većih datoteka. Takva organizacija može biti dovoljna za jednostavne projekte, ali s povećanjem broja funkcionalnosti otežava testiranje, održavanje i pronalaženje pogrešaka. Podjela na module s jasno definiranim odgovornostima zato je prikladnija za daljnji razvoj sustava.

4.2 Komunikacija između komponenti sustava

Radi prikaza načina povezivanja mobilne aplikacije, poslužiteljskog dijela, baze podataka i vanjskog AI servisa izrađen je arhitekturni prikaz razvijenog sustava, prikazan na slici 17.

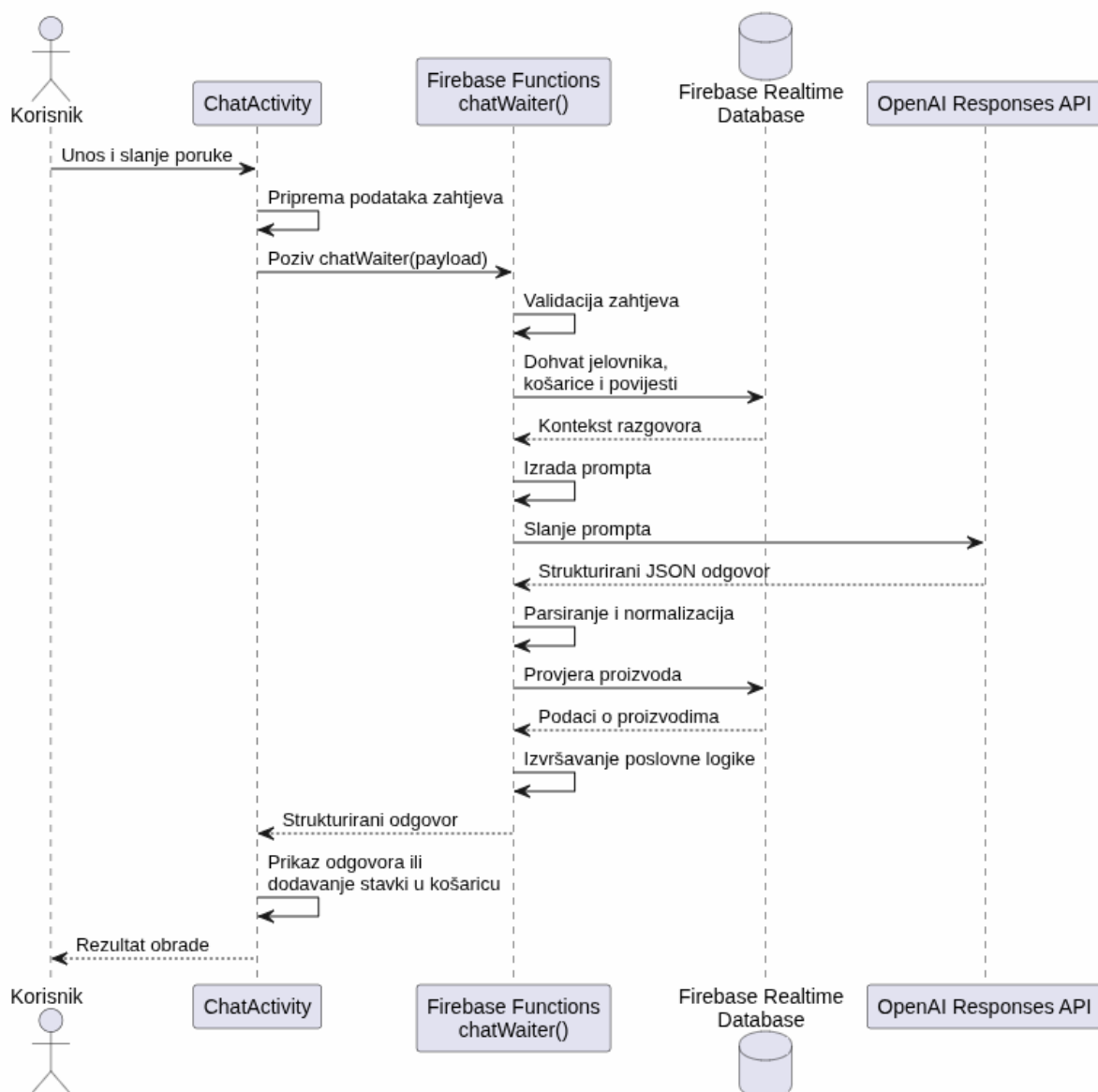


Slika 17. Arhitektura razvijenog sustava i komunikacija između njegovih glavnih komponenti. Izvor: izrada autora.

Slika 17 prikazuje arhitekturu razvijenog sustava i međusobnu komunikaciju njegovih glavnih komponenti. Korisnik komunicira s Android mobilnom aplikacijom, koja putem Firebase SDK-a poziva funkciju `chatWaiter()`. Ta funkcija predstavlja središnje mjesto poslovne logike te prema potrebi dohvaća podatke iz Firebase Realtime Database baze podataka i komunicira s OpenAI API-jem radi obrade korisničkog zahtjeva. Nakon završetka obrade poslužiteljski dio vraća strukturirani odgovor koji mobilna aplikacija prikazuje korisniku. Takva arhitektura jasno razdvaja korisničko sučelje, poslovnu logiku i vanjske servise, čime se olakšavaju održavanje i buduće proširenje funkcionalnosti, dok se osjetljivi podaci, poput API ključa, zadržavaju na poslužiteljskoj strani.

Arhitekturni prikaz daje pregled glavnih komponenti sustava i njihovih međusobnih veza, dok je za detaljniji prikaz redoslijeda razmjene poruka izrađen UML sekvencijski dijagram. Dijagram prikazuje tijek obrade korisničkog zahtjeva od trenutka unosa poruke do prikaza

konačnog odgovora u mobilnoj aplikaciji.



Slika 18. UML sekvencijski dijagram obrade korisničkog zahtjeva. Izvor: izrada autora.

Slika 18 prikazuje redoslijed komunikacije između korisnika, aktivnosti ChatActivity, funkcije chatWaiter(), baze Firebase Realtime Database i OpenAI Responses API-ja. Nakon što korisnik pošalje poruku, mobilna aplikacija priprema podatke zahtjeva i prosljeđuje ih poslužiteljskoj funkciji. Funkcija provjerava zahtjev, dohvaća potreban kontekst iz baze podataka te na temelju prikupljenih informacija izrađuje prompt koji se šalje OpenAI Responses API-ju. Dobiveni strukturirani odgovor zatim se parsira i normalizira, nakon čega se provjeravaju proizvodi i izvršava odgovarajuća poslovna logika. Konačni rezultat vraća se mobilnoj aplikaciji, koja korisniku prikazuje odgovor ili dodaje odgovarajuće stavke u košaricu.

4.3 Komunikacija s modelima umjetne inteligencije

Komunikacija s modelima umjetne inteligencije izdvojena je u direktorij ai, koji sadrži module za povezivanje s vanjskim AI servisima, analizu korisničkih zahtjeva i planiranje tijeka razgovora. Takvim razdvajanjem poslovna logika aplikacije nije izravno vezana uz pojedini model, što olakšava održavanje i moguću zamjenu pružatelja usluge.

Za komunikaciju s OpenAI servisom koristi se modul openai.js, u kojem je definirana funkcija callOpenAI(). Funkcija prima API ključ i pripremljeni prompt, šalje zahtjev prema OpenAI Responses API-ju koristeći model gpt-5-mini te iz primljenog odgovora izdvaja generirani sadržaj [17].

Model gpt-5-mini odabran je kao prikladno rješenje za potrebe razvijenog sustava, uzimajući u obzir složenost korisničkih zahtjeva i troškove korištenja API-ja [18]. Model se prvenstveno koristi za obradu zahtjeva povezanih s jelovnikom, preporukama i sastavljanjem narudžbe, dok se dobiveni odgovori dodatno provjeravaju i obrađuju u poslužiteljskoj logici prije prosljeđivanja mobilnoj aplikaciji.

API ključ pohranjen je kao Firebase tajna i dostupan je isključivo poslužiteljskom dijelu sustava. Mobilna aplikacija zato ne komunicira izravno s OpenAI API-jem i ne izlaže pristupne podatke korisniku. Takva arhitektura omogućuje i promjenu modela bez potrebe za izmjenama mobilne aplikacije.

4.4 Planiranje razgovora i prepoznavanje korisničke namjere

Za razumijevanje korisničkih poruka napisanih prirodnim jezikom koristi se veliki jezični model koji određuje značenje zahtjeva i planira daljnji tijek razgovora. Konverzacijski preporučeni sustavi omogućuju prikupljanje korisničkih zahtjeva kroz dijalog te prilagođavanje preporuka informacijama dobivenima tijekom interakcije [25]. Obrada je u razvijenom sustavu organizirana u više modula unutar direktorija ai, pri čemu svaki modul ima zasebnu odgovornost.

Planiranje razgovora implementirano je u modulu conversationPlanner.js. Modul izrađuje prompt koji, osim korisničke poruke, sadrži podatke o digitalnom jelovniku, trenutačnoj košarici, prethodnim porukama i pravilima ponašanja modela. Uključivanjem konteksta model može preciznije protumačiti kratke ili nejasne poruke koje se odnose na prethodni tijek razgovora.

Alternativno bi se modelu mogla poslati samo posljednja korisnička poruka. Time bi se smanjila količina ulaznih podataka i potrošnja tokena, ali bi model imao manje informacija o dostupnim proizvodima i trenutnom stanju narudžbe. Odabranim pristupom modelu se pruža širi kontekst potreban za razumijevanje zahtjeva, uz veću količinu ulaznih podataka.

Promptom se definiraju uloga modela, podržane korisničke namjere i pravila za prepoznavanje proizvoda. Jasno definiranje uputa, konteksta i očekivanog oblika odgovora predstavlja važan dio oblikovanja prompta jer omogućuje usmjeravanje ponašanja velikog jezičnog modela prema zahtjevima konkretne aplikacije [19]. Od modela se zahtijeva vraćanje odgovora u strukturiranom JSON formatu, umjesto u obliku slobodno generiranog teksta. Time se pojednostavljuje programska obrada rezultata i omogućuje njegovo povezivanje s daljnjom poslovnom logikom sustava.

Isječak programskog koda 5. Definiranje prompta za planiranje razgovora

```
const prompt = `
Ti si AI konobar u restoranu.
Tvoj posao:
- Razumjeti korisnikovu poruku.
- Voditi narudžbu kroz razgovor.

- Vratiti SAMO JSON.
- Ne izmišljati artikle koji ne postoje u meniju.
Mogući intenti:
- menu_question
- category_question
- add_to_cart
```

U isječku programskog koda 5 prikazan je dio prompta kojim se određuju uloga modela, način odgovaranja i skup podržanih korisničkih namjera. Modelu se također nalaže da ne predlaže proizvode koji ne postoje u jelovniku te da rezultat vrati isključivo u JSON formatu, čime se olakšava njegova daljnja obrada u poslužiteljskoj logici.

Nakon primitka odgovora modul `conversationPlanner.js` uklanja moguće oznake formatiranja, provjerava valjanost JSON strukture i pretvara sadržaj u programski objekt. Odgovor može sadržavati korisničku namjeru, kategoriju proizvoda, prepoznate artikle, razinu pouzdanosti, broj osoba i informaciju o potrebi za dodatnim pojašnjenjem.

Veliki jezični model pritom služi za razumijevanje poruke i izradu strukturiranog plana obrade. Poslovne operacije, poput pretraživanja jelovnika, upravljanja košaricom, dohvaćanja podataka i generiranja preporuka, ostaju u sustavu Firebase Functions. Takvom podjelom smanjuje se mogućnost da model samostalno izvrši nepredviđenu radnju te se

zadržava programska kontrola nad procesom naručivanja.

4.5 Normalizacija i obrada strukturiranog plana razgovora

Nakon obrade korisničke poruke odgovor modela pretvara se u oblik pogodan za daljnju obradu u poslužiteljskom dijelu sustava. Budući da odgovor može sadržavati dodatne oznake formatiranja ili vrijednosti neočekivanog tipa, najprije se izdvaja JSON sadržaj i pretvara u JavaScript objekt.

Funkcija `normalizeConversationPlan()` provjerava pripada li prepoznata korisnička namjera dopuštenom skupu vrijednosti. Ako model vrati nepoznatu namjeru, ona se zamjenjuje vrijednošću `unknown`. Provjeravaju se i normaliziraju podaci o kategoriji, artiklima, količini, broju narudžbe, broju osoba, razini pouzdanosti i potrebi za dodatnim pojašnjenjem.

Svaki prepoznati artikl pretvara se u ujednačen objekt koji sadrži naziv, kategoriju, količinu i moguću napomenu. Brojčane vrijednosti pretvaraju se u odgovarajući brojčani tip, dok se tekstualna polja provjeravaju prije daljnje obrade. Time ostatak sustava dobiva dosljednu strukturu podataka neovisno o manjim razlikama u odgovoru modela. Dio postupka provjere i normalizacije strukturiranog plana prikazan je u isječku programskog koda 6.

Isječak programskog koda 6. Normalizacija strukturiranog plana razgovora

```
function normalizeConversationPlan(parsed) {
  const intent = allowedIntents.includes(parsed.intent)
    ? parsed.intent
    : "unknown";

  const items = Array.isArray(parsed.items)
    ? parsed.items.map((item) => ({
      name: typeof item.name === "string" ? item.name : "",
      quantity: Number(item.quantity || 1),
      note: typeof item.note === "string" ? item.note : "",
    }))
    : [];

  return {
    intent,
    category: typeof parsed.category === "string"
      ? parsed.category
      : "",
    items,
  };
}
```

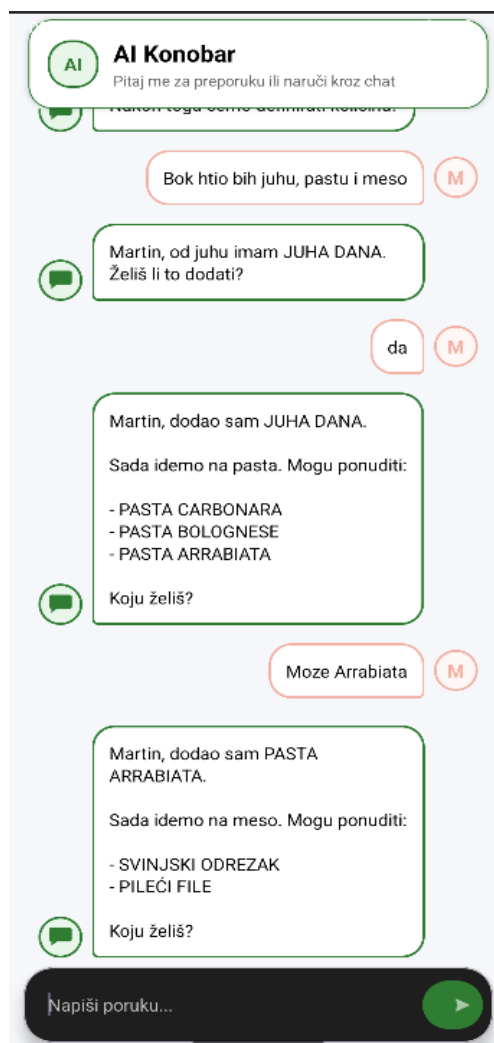
U isječku programskog koda 6 prikazane su provjera korisničke namjere i normalizacija podataka dobivenih od modela. Nepoznata korisnička namjera zamjenjuje se vrijednošću unknown, dok se ostala polja pretvaraju u očekivane tipove i strukture prije njihove daljnje obrade. Ako odgovor nije moguće pretvoriti u valjan JSON objekt ili ne sadrži očekivanu strukturu, poziva se funkcija `createUnknownPlan()`, koja vraća zadani plan s namjerom unknown.

Rezervni mehanizam omogućuje obradu neočekivanog odgovora modela bez prekida uobičajenog tijeka obrade zahtjeva. Sustav umjesto toga može zatražiti od korisnika preciznije oblikovanje zahtjeva. Takva obrada povećava otpornost AI funkcionalnosti na neočekivane ili nepotpune odgovore modela.

4.6 Obrada korisničkih zahtjeva i generiranje preporuka

Nakon prepoznavanja korisničke namjere poslužiteljska logika određuje koji se modul treba izvršiti i koje podatke treba vratiti mobilnoj aplikaciji. Ovisno o zahtjevu, sustav može pretražiti jelovnik, prikazati kategoriju proizvoda, generirati preporuku, dohvatiti sadržaj košarice ili prethodne narudžbe te pokrenuti postupno sastavljanje obroka.

Postupno sastavljanje obroka implementirano je u direktoriju `mealBuilder`. Kada korisnik navede željene dijelove obroka, primjerice juhu, tjesteninu i mesno jelo, sustav izrađuje plan koji sadrži tražene kategorije, trenutni korak, prethodne odabire i preskočene kategorije. Sustav za svaki korak iz jelovnika dohvaća odgovarajuće proizvode koji se razvrstavaju prema popularnosti, a korisniku se prikazuje ograničen broj prijedloga. Nakon korisnikova odabira plan se ažurira i razgovor prelazi na sljedeći dio obroka.



Slika 19. Postupno sastavljanje obroka s pomoću AI asistenta. Izvor: izrada autora.

Na prikazanom primjeru na slici 19 korisnik traži obrok koji uključuje juhu, tjesteninu i meso. Sustav pamti trenutačni korak, ponuđene proizvode i korisnikove odabire u metapodacima razgovora, čime je omogućeno vođenje procesa kroz više poruka.

Moduli iz direktorija recommendations generiraju dodatne prijedloge na temelju sadržaja košarice i nedavno odabranih proizvoda. Primjerice, uz glavno jelo mogu se predložiti piće ili desert, dok se već odabrani proizvodi uklanjaju iz skupa prijedloga kako bi se izbjeglo njihovo ponavljanje.

Preporučeni sustavi koriste informacije o korisnicima, njihovim prethodnim interakcijama i dostupnim proizvodima kako bi izdvojili stavke koje bi korisniku mogle biti relevantne [20]. U razvijenom sustavu osobne preporuke temelje se na prethodno dovršenim narudžbama korisnika, dok se popularni proizvodi određuju prema učestalosti naručivanja svih korisnika. Ako nema dovoljno podataka, sustav koristi proizvode iz odgovarajuće kategorije kao

rezervni prijedlog. Pri pretraživanju se mogu uzeti u obzir i prehrambeni zahtjevi, poput vegetarijanske prehrane ili izbjegavanja glutena, mlijeka, orašastih plodova i pojedinih sastojaka.

Moduli history i repeat omogućuju obradu poruka koje se oslanjaju na prethodni kontekst. Sustav pamti posljednje ponuđene proizvode, pa može protumačiti kratke odgovore poput „dodaj prvi” ili „uzmi to”. Korisnik također može dohvatiti prethodnu narudžbu i ponovno dodati njezine stavke u košaricu.

Na kraju obrade modul responses oblikuje jedinstven strukturirani odgovor koji sadrži tekstualnu poruku, vrstu odgovora, proizvode, dostupne radnje i metapodatke potrebne za nastavak razgovora. Takva organizacija omogućuje da mobilna aplikacija ostane zadužena za prikaz i korisničku interakciju, dok se pravila preporučivanja i upravljanja razgovorom mogu mijenjati na poslužiteljskoj strani.

Odabrani pristup omogućuje prirodnu i kontekstualno prilagođenu komunikaciju, ali ovisi o kvaliteti podataka o jelovniku, prethodnim narudžbama i ispravnom prepoznavanju korisničke namjere. Kod novih korisnika ili nedovoljno jasno oblikovanih zahtjeva preporuke mogu biti manje personalizirane, zbog čega sustav koristi rezervne prijedloge i mogućnost dodatnog pojašnjenja.

4.7 Obrada korisničkog zahtjeva

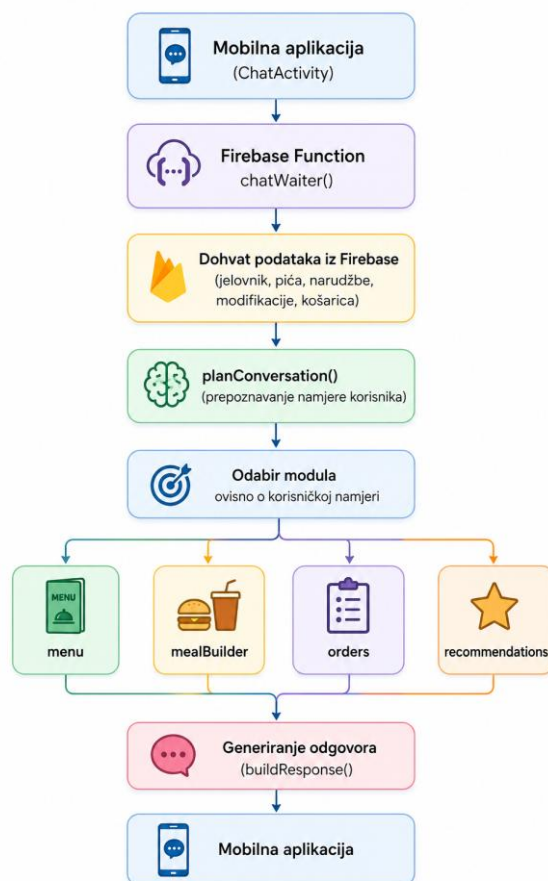
Nakon što korisnik pošalje poruku iz aktivnosti ChatActivity, mobilna aplikacija poziva Firebase funkciju chatWaiter(), koja predstavlja ulaznu točku poslužiteljskog dijela sustava. Poslovna logika i komunikacija s modelom umjetne inteligencije izvršavaju se na poslužiteljskoj strani, dok je mobilna aplikacija zadužena za unos i prikaz podataka.

Funkcija chatWaiter() iz baze Firebase Realtime Database dohvaća podatke potrebne za obradu zahtjeva, poput digitalnog jelovnika, sadržaja košarice, prethodnih narudžbi i povijesti razgovora. Nakon toga provjerava odnosi li se zahtjev na već započeti proces, primjerice nastavak sastavljanja obroka ili ponavljanje prethodne narudžbe. Ako poseban slučaj nije prepoznat, poziva se funkcija planConversation(), koja određuje korisničku namjeru i oblikuje strukturirani plan obrade.

Na temelju prepoznate namjere zahtjev se usmjerava odgovarajućem modulu za obradu jelovnika, narudžbi, preporuka ili tijeka razgovora. Nakon završetka obrade funkcija buildResponse() oblikuje odgovor koji može sadržavati tekstualnu poruku, proizvode,

dostupne radnje i metapodatke potrebne za nastavak razgovora.

Centralizirana obrada na poslužiteljskoj strani omogućuje jednostavnije održavanje poslovnih pravila i njihovu primjenu bez potrebe za izmjenama mobilne aplikacije. Tijek obrade korisničkog zahtjeva kroz glavne komponente sustava prikazan je na slici 20.



Slika 20. Tijek obrade korisničkog zahtjeva unutar poslužiteljskog dijela sustava. Izvor: izrada autora.

Slika 20 prikazuje redoslijed obrade od primitka korisničke poruke do vraćanja strukturiranog odgovora mobilnoj aplikaciji. Funkcija `chatWaiter()` koordinira rad specijaliziranih modula, dok su prepoznavanje namjere, pristup podacima i izvršavanje poslovnih pravila međusobno odvojeni. Takva organizacija olakšava izmjenu pojedinih dijelova sustava, primjerice načina prepoznavanja namjere ili generiranja preporuka, bez značajnih promjena ostalih modula i mobilne aplikacije.

4.8 Projektiranje prompta

Veliki jezični modeli kvalitetu odgovora uvelike temelje na sadržaju i strukturi prompta koji im se prosljeđuje. Jasno definiranje konteksta, uloge modela, očekivanog oblika odgovora i pravila ponašanja može povećati dosljednost rezultata i olakšati primjenu modela u konkretnom programskom sustavu [19]. Zbog toga je tijekom razvoja AI asistenta posebna pažnja posvećena projektiranju prompta kako bi model mogao pouzdano prepoznati korisničke namjere i vratiti odgovor prikladan za daljnju programsku obradu.

Razvoj prompta proveden je iterativnim pristupom. Tijekom razvoja isprobano je više verzija prompta koje su se razlikovale prema količini dostupnog konteksta, načinu definiranja pravila te obliku očekivanog odgovora. U početnim verzijama prompt je sadržavao uglavnom korisničku poruku i osnovni opis zadatka. Takav je pristup u pojedinim slučajevima rezultirao odgovorima koji nisu bili dovoljno dosljedni ili su sadržavali informacije koje nije bilo moguće jednostavno obraditi unutar poslužiteljskog dijela sustava.

Kako bi se povećala pouzdanost rada, konačna verzija prompta proširena je dodatnim informacijama. Osim korisničke poruke, modelu se prosljeđuju podaci o digitalnom jelovniku, trenutačnoj košarici, prethodnom tijeku razgovora te pravilima koja određuju način prepoznavanja korisničkih namjera i oblik odgovora. Definiranjem jasne uloge modela, mogućih korisničkih zahtjeva i zahtjeva da odgovor bude vraćen isključivo u JSON formatu smanjena je mogućnost generiranja odgovora koji nisu usklađeni s poslovnom logikom aplikacije.

Odabran je pristup u kojem veliki jezični model služi prvenstveno za razumijevanje korisničkog zahtjeva i planiranje daljnjeg tijeka razgovora, dok se sva poslovna pravila, dohvat podataka iz baze, razne akcije i generiranje konačnih odgovora izvršavaju unutar Firebase Functions sustava.

Kvaliteta pojedinih verzija prompta procjenjivana je tijekom razvoja kroz funkcionalno testiranje različitih korisničkih scenarija. Kao kriteriji uspješnosti promatrani su ispravnost prepoznavanja korisničke namjere, valjanost generiranog JSON odgovora te mogućnost uspješnog izvršavanja odgovarajuće poslovne logike na poslužiteljskoj strani. Na temelju dobivenih rezultata prompt je postupno nadograđivan dodavanjem konteksta i preciznijih pravila za generiranje odgovora. Reprezentativni dijelovi promptova korištenih u radu AI asistenta prikazani su u Dodatku A.

4.9 Odabir modela GPT-5-mini

Za razvoj AI asistenta odabran je model GPT-5-mini. Pri odabiru modela uzeti su u obzir zahtjevi razvijenog sustava, koji uključuju razumijevanje korisničkih poruka napisanih prirodnim jezikom, prepoznavanje korisničke namjere, korištenje konteksta prethodnog razgovora i generiranje strukturiranog odgovora prikladnog za daljnju programsku obradu. Budući da se veliki jezični model u razvijenom sustavu prvenstveno koristi za razumijevanje zahtjeva i izradu strukturiranog plana obrade, dok se poslovna pravila izvršavaju na poslužiteljskoj strani, pri odabiru modela bilo je potrebno uzeti u obzir ispravnost obrade korisničkog zahtjeva, vrijeme odziva i trošak izvođenja.

Kako bi se odabir modela dodatno evaluirao, provedena je usporedba pet modela dostupnih putem OpenAI API-ja: GPT-5-mini, GPT-5-nano, GPT-4.1-mini, GPT-4.1-nano i GPT-5 [18]. Evaluacija je podijeljena u dvije skupine. Prvu skupinu činili su osnovni i uglavnom jednoznačni korisnički zahtjevi, dok je druga skupina obuhvatila složenije zahtjeve koji zahtijevaju korištenje prethodnog konteksta, povezivanje više informacija unutar istog zahtjeva i održavanje višekoračnog tijeka razgovora.

U prvoj skupini korišteno je pet reprezentativnih scenarija: dodavanje proizvoda i količine, dodavanje proizvoda s modifikacijom, prikaz proizvoda određene kategorije, generiranje preporuke te obrada zahtjeva za proizvodom koji nije dostupan u digitalnom jelovniku. Svaki scenarij izveden je pet puta za svaki model, čime je ukupno provedeno 125 izvođenja.

Modeli su uspoređeni prema ispravnosti obrade korisničkog zahtjeva, vremenu odziva i procijenjenom trošku izvođenja. Vrijeme odziva mjereno je od slanja zahtjeva funkciji `chatWaiter()` do primitka konačnog odgovora mobilnoj aplikaciji, dok su podaci o potrošnji tokena i procijenjenom trošku prikupljeni iz podataka koje vraća OpenAI Responses API.

Kako bi uvjeti usporedbe bili jednaki za sve modele, za svaki je model korišten isti skup korisničkih poruka i isti kontekst koji se proslijeđuje AI asistentu. Ispravnost obrade određena je usporedbom dobivenog rezultata s unaprijed očekivanim ponašanjem sustava za pojedini scenarij. Za vrijeme odziva izračunate su aritmetička sredina i uzoračka standardna devijacija svih izvođenja pojedinog modela, dok je prosječni trošak određen na temelju potrošnje tokena tijekom testiranja. Rezultati prve skupine prikazani su u tablici 2.

Model	Ispravnost (%)	Vrijeme odziva ($\bar{x} \pm SD$)	Prosječni trošak po zahtjevu (USD)
GPT-5-mini	100	11,621 $\pm$ 7,561 s	0,002035
GPT-5-nano	100	11,574 $\pm$ 7,060 s	0,000676
GPT-4.1-mini	100	2,853 $\pm$ 1,275 s	0,002903
GPT-4.1-nano	95	2,661 $\pm$ 1,704 s	0,000615
GPT-5	100	15,928 $\pm$ 11,042 s	0,012067

Tablica 2. Usporedba evaluiranih modela na osnovnim scenarijima. Izvor: izrada autora.

Pri izračunu ispravnosti uspoređeni su scenariji u kojima je bilo moguće jasno utvrditi očekivani rezultat obrade korisničkog zahtjeva. Iako je prva skupina obuhvatila pet scenarija, scenarij generiranja preporuke nije uključen u izračun postotka ispravnosti jer je kod svih testiranih modela uočeno jednako ponašanje povezano s postojećom poslovnom logikom sustava. Ispravnost je stoga izračunata na temelju četiri od pet evaluiranih scenarija, odnosno ukupno 20 izvođenja po modelu.

Rezultati osnovnih scenarija pokazali su relativno male razlike u ispravnosti između evaluiranih modela. GPT-5-mini, GPT-5-nano, GPT-4.1-mini i GPT-5 ostvarili su potpunu ispravnost u izvođenjima uključenima u izračun, dok je GPT-4.1-nano ostvario 95 % ispravnosti. GPT-4.1-mini posebno se istaknuo kratkim vremenom odziva te je na temelju samo tih rezultata predstavljao vrlo konkurentan izbor. S druge strane, GPT-5 imao je najveće prosječno vrijeme odziva i znatno veći procijenjeni trošak izvođenja. Rezultati prve skupine stoga nisu omogućili dovoljno jasno razlikovanje modela prema sposobnosti obrade složenijih razgovora.

Zbog toga je provedena dodatna evaluacija u kojoj su ponovno uključena svih pet modela. Cilj druge skupine nije bio ponovno provjeravati jednostavne naredbe, nego ispitati ponašanje modela u situacijama koje su bliže stvarnom višekoračnom razgovoru korisnika s AI asistentom. Korišteno je pet složenijih scenarija, a svaki je scenarij izveden pet puta za svaki model. Time je provedeno dodatnih 125 izvođenja, odnosno 25 izvođenja po modelu.

Za svaki složeniji scenarij unaprijed je definirano očekivano ponašanje sustava. Izvođenje je označeno ispravnim samo ako je model pravilno prepoznao korisnički zahtjev, iskoristio potreban kontekst prethodnog razgovora i vratio rezultat koji omogućuje nastavak predviđenog tijeka obrade. Ako je model izgubio prethodni kontekst, pogrešno povezao korisničku referencu, nepravilno razdvojio elemente zahtjeva ili prekinuo očekivani višekoračni tijek razgovora, izvođenje je označeno neispravnim.

Složeniji scenariji obuhvatili su korištenje prethodnog konteksta razgovora, razrješavanje referenci na prethodno prikazane proizvode, istodobnu obradu različitih elemenata jednog zahtjeva te održavanje započetog tijeka razgovora kroz više korisničkih poruka.

Jedan od scenarija odnosio se na korištenje prethodnog konteksta. Nakon što je asistent korisniku prethodno prikazao određeni sendvič, korisnik je u sljedećoj poruci napisao „Daj mi taj sendvič.” Očekivano ponašanje bilo je povezati izraz „taj sendvič” s prethodno prikazanim proizvodom. Ponovno prikazivanje cijele kategorije sendviča ili zanemarivanje prethodnog konteksta smatralo se neispravnim rezultatom.

Drugi scenarij provjeravao je sposobnost pravilnog razdvajanja službene modifikacije proizvoda od slobodne korisničke napomene. Korisnički zahtjev glasio je „Daj mi ledenu kavu s pistacijom i mrvicama keksa.” U digitalnom jelovniku za ledenu kavu bila je definirana službena modifikacija „PISTACIJA OKUS”, dok mrvice keksa nisu postojale među dostupnim modifikacijama. Očekivano ponašanje bilo je spremiti „PISTACIJA OKUS” kao modifikaciju proizvoda, a zahtjev za mrvicama keksa kao korisničku napomenu.

Dodatno je testirano postupno sastavljanje obroka započeto zahtjevom poput „Želim juhu, tjesteninu i neko meso.” U tom je slučaju model kroz više uzastopnih poruka morao zadržati informaciju o već odabranim dijelovima obroka i nastaviti započeti tijek razgovora sve dok nisu obrađene sve tražene kategorije. Četvrti scenarij uključivao je odabir proizvoda iz prethodno prikazane liste korištenjem rednog broja, dok je peti obuhvatio kombiniranje više zahtjeva u istoj poruci uz zadržavanje prethodnog konteksta razgovora.

Rezultati druge skupine pokazali su znatno veće razlike između modela nego osnovni scenariji. GPT-5-mini ispravno je obradio svih 25 izvođenja. GPT-5-nano ostvario je 20 ispravnih rezultata od ukupno 25 izvođenja, odnosno 80 % ispravnosti. GPT-5 također je ostvario 80 % ispravnosti, ali uz nepovoljnije vrijeme odziva i veći trošak izvođenja. GPT-4.1-mini ostvario je ispravan rezultat u 20 % izvođenja. Kod njega su se, unatoč kratkom vremenu odziva, češće pojavljivale poteškoće u korištenju prethodnog konteksta, razrješavanju referenci i održavanju višekoračnog tijeka razgovora. Slično ponašanje zabilježeno je i kod modela GPT-4.1-nano, koji je također ostvario 20 % ispravnosti u složenijim scenarijima. Iako su modeli GPT-4.1 obitelji ostvarivali kraće vrijeme odziva, u dodatnoj evaluaciji pokazali su slabiju pouzdanost pri zahtjevima koji su ovisili o prethodnom kontekstu i višekoračnom tijeku razgovora.

Model	Broj izvođenja	Ispravnost (%)
GPT-5-mini	25	100
GPT-5-nano	25	80
GPT-4.1-mini	25	20
GPT-4.1-nano	25	20
GPT-5	25	80

Tablica 3. Usporedba evaluiranih modela na složenijim scenarijima. Izvor: izrada autora.

Rezultati dodatne evaluacije posebno su važni za usporedbu modela GPT-5-mini i GPT-5-nano. GPT-5-nano ostvario je najniži prosječni trošak među modelima GPT-5 obitelji uključenima u osnovnu evaluaciju, ali se u složenijim scenarijima pokazala razlika u ispravnosti. GPT-5-nano ostvario je 80 % ispravnosti, dok je GPT-5-mini ispravno obradio svih 25 izvođenja. U kontekstu razvijene aplikacije prednost je stoga dana većoj pouzdanosti GPT-5-mini modela pri obradi složenijih i kontekstualno ovisnih zahtjeva.

Sličan zaključak proizlazi iz usporedbe s modelima GPT-4.1. Njihova prednost prvenstveno je bila kratko vrijeme odziva, što je posebno vidljivo u rezultatima osnovne evaluacije. Međutim, u složenijim scenarijima modeli GPT-4.1-mini i GPT-4.1-nano pokazali su poteškoće kada odgovor nije ovisio samo o trenutačnoj korisničkoj poruci, nego i o prethodnom kontekstu i stanju višekoračnog razgovora. Zbog toga kraće vrijeme odziva nije bilo dovoljno za njegov odabir u konačnoj verziji aplikacije.

GPT-5 ostvario je veću ispravnost u složenijim scenarijima od GPT-4.1-mini, ali u provedenoj evaluaciji nije ostvario veću ispravnost od GPT-5-mini. Uz to je već u osnovnoj skupini imao najveće prosječno vrijeme odziva od 15,928 sekundi i prosječni trošak od 0,012067 USD po zahtjevu, što je približno šest puta veći trošak od GPT-5-mini.

Na temelju obje skupine testiranja GPT-5-mini zadržan je kao model korišten u konačnoj verziji sustava. GPT-4.1-mini i GPT-4.1-nano pružali su kraće vrijeme odziva, ali su se pokazali manje pouzdanima u složenijim i kontekstualno ovisnim scenarijima. GPT-5-nano bio je troškovno povoljniji, ali je u dodatnoj evaluaciji ostvario nižu ispravnost. GPT-5 zahtijevao je više vremena i imao znatno veći trošak izvođenja bez povećanja ispravnosti u odnosu na GPT-5-mini. GPT-5-mini tako se u okviru provedene evaluacije pokazao kao najprikladniji kompromis između ispravnosti obrade složenijih zahtjeva, sposobnosti održavanja konteksta razgovora, vremena odziva i troška izvođenja.

4.10 Ograničenja primijenjenog rješenja

Iako primjena velikih jezičnih modela omogućuje prirodnu komunikaciju s korisnikom i jednostavniji proces naručivanja, konverzijski preporučeni sustavi ovise o sposobnosti sustava da pravilno razumije korisničke zahtjeve i kontekst razgovora [25]. Razvijeno rješenje stoga ima određena ograničenja, osobito u slučajevima kada korisnik koristi dvosmislene izraze, nepotpune rečenice ili nazive proizvoda koji nisu jednoznačno definirani u digitalnom jelovniku.

Dodatno ograničenje predstavlja mogućnost generiranja netočnih ili izmišljenih informacija, odnosno pojava halucinacija kod velikih jezičnih modela [26]. Kako bi se smanjio utjecaj takvih situacija, model u razvijenom sustavu ne izvršava poslovnu logiku samostalno. Njegova je uloga prepoznati korisničku namjeru i predložiti daljnje korake, dok se provjera proizvoda, upravljanje košaricom i dohvat podataka iz baze izvode na poslužiteljskoj strani. Time se smanjuje mogućnost dodavanja nepostojećih proizvoda ili izvršavanja radnji koje

nisu predviđene poslovnom logikom sustava.

Jedno od ograničenja razvijenog rješenja jest i ovisnost o vanjskim servisima i mrežnoj vezi. Vrijeme odziva sustava ovisi o komunikaciji s OpenAI API-jem, izvršavanju funkcija unutar Firebase Functions okruženja te kvaliteti mrežne veze. Nedostupnost ili povećano opterećenje pojedinog vanjskog servisa može stoga utjecati na vrijeme potrebno za obradu korisničkog zahtjeva.

U budućem razvoju sustava moguće je dodatno unaprijediti AI asistenta proširenjem postojećeg pristupa personalizaciji, primjerice uključivanjem većeg broja podataka o prethodnim odabirima i narudžbama korisnika. Sustav bi se mogao dodatno evaluirati kroz testiranje s većim brojem stvarnih korisnika i većim skupom različitih korisničkih zahtjeva. Provedena usporedba velikih jezičnih modela također bi se mogla proširiti uključivanjem dodatnih modela, složenijih korisničkih scenarija i većeg broja ponavljanja kako bi se detaljnije ispitala stabilnost i kvaliteta rada AI asistenta.

4.11 Potrošnja tokena i procjena troškova

Budući da se korištenje modela putem OpenAI API-ja naplaćuje prema količini obrađenih ulaznih i izlaznih tokena, tijekom evaluacije provedeno je mjerenje potrošnje tokena korištenjem podataka koje vraća OpenAI Responses API [17]. Za svaki poziv evidentirani su broj ulaznih i izlaznih tokena, ukupan broj tokena te broj predmemoriranih (engl. cached) ulaznih tokena. Procijenjeni trošak izvođenja izračunat je na temelju cijena korištenog modela, pri čemu su predmemorirani ulazni tokeni obračunati prema pripadajućoj nižoj cijeni [27].

Kako bi se smanjio utjecaj varijabilnosti pojedinačnog izvođenja, svaki od pet odabranih scenarija izvršen je pet puta korištenjem modela GPT-5-mini. Za svaki scenarij izračunate su aritmetička sredina i uzoračka standardna devijacija potrošnje tokena i procijenjenog troška. U slučajevima u kojima je obrada jednog korisničkog zahtjeva zahtijevala više poziva modelu, potrošnja pojedinačnih poziva najprije je zbrojena na razini korisničkog zahtjeva, a zatim su izračunate opisne statistike. Rezultati mjerenja potrošnje tokena i procijenjenog troška za odabrane scenarije prikazani su u tablici 3.

Scenarij	Ponavljanja	Ukupno tokena ($\bar{x} \pm SD$)	Trošak u USD ($\bar{x} \pm SD$)
Dodavanje dvije Coca- Cole	5	4368,6 $\pm$ 34,4	0,000557 $\pm$ 0,000069
Dodavanje pizze capricciose bez gljiva	5	4378,0 $\pm$ 35,6	0,000565 $\pm$ 0,000071
Prikaz dostupnih pizza	5	12387,2 $\pm$ 176,1	0,002639 $\pm$ 0,000948
Preporuka pića uz pizzu	5	12912,2 $\pm$ 242,8	0,003168 $\pm$ 0,000486
Upit za nedostupan proizvod	5	13120,2 $\pm$ 328,7	0,003246 $\pm$ 0,000481

Tablica 4. Potrošnja tokena i procijenjeni trošak izvođenja scenarija. Izvor: izrada autora.

Na temelju provedenih mjerenja prosječna potrošnja u promatranih pet scenarija iznosila je približno 9433 tokena po korisničkom zahtjevu. Pritom je vidljiva razlika između jednostavnijih zahtjeva, poput dodavanja proizvoda u košaricu, i složenijih zahtjeva koji zahtijevaju obradu većeg konteksta razgovora. Na procijenjeni trošak pojedinog izvođenja dodatno je utjecalo korištenje predmemoriranih ulaznih tokena, koji se obračunavaju po nižoj cijeni od standardnih ulaznih tokena [27].

Rezultati pokazuju da složeniji scenariji zahtijevaju znatno veći broj tokena. Razlog tome je što se modelu, osim korisničke poruke, prosljeđuje i dodatni kontekst potreban za razumijevanje razgovora, uključujući dio povijesti komunikacije, stanje košarice te informacije o dostupnim artiklima. Takav pristup omogućuje modelu korištenje konteksta potrebnog za obradu korisničkog zahtjeva, ali istodobno povećava broj obrađenih tokena i ukupni trošak izvođenja.

Kao moguće buduće poboljšanje predlaže se smanjenje količine konteksta koji se prosljeđuje modelu te obrada jednostavnijih operacija, poput prikaza sadržaja košarice ili izmjene količine već odabranih proizvoda, izravno na poslužiteljskoj strani bez pozivanja velikog jezičnog modela. Time bi se smanjila potrošnja tokena, skratilo vrijeme odziva i smanjili operativni troškovi sustava.

5 Testiranje sustava

Testiranje razvijenog sustava provedeno je kako bi se provjerila ispravnost osnovnih funkcionalnosti mobilne aplikacije, komunikacije s poslužiteljskim dijelom i ponašanja AI asistenta. Funkcionalno testiranje obuhvatilo je prijavu korisnika, skeniranje QR koda, dohvat digitalnog jelovnika, sastavljanje i slanje narudžbe, praćenje njezina statusa te rad kuhinjskog dijela aplikacije. AI asistent dodatno je evaluiran kroz različite vrste korisničkih zahtjeva povezanih s pretraživanjem jelovnika, upravljanjem narudžbom i korištenjem konteksta razgovora.

5.1 Funkcionalno testiranje mobilne aplikacije

Funkcionalno testiranje usmjereno je na provjeru ponašanja programskog sustava u odnosu na definirane funkcionalne zahtjeve, pri čemu se izvođenjem testnih slučajeva uspoređuju očekivani i dobiveni rezultati [28]. Rezultati funkcionalnog testiranja mobilne aplikacije prikazani su u tablici 4. Testiranjem su obuhvaćene ključne funkcionalnosti aplikacije, uključujući anonimnu prijavu korisnika, skeniranje QR koda, dohvat digitalnog jelovnika, prilagodbu proizvoda, slanje narudžbe i praćenje njezina statusa. Za svaki testni scenarij navedeni su očekivani i dobiveni rezultati kako bi se provjerila usklađenost stvarnog ponašanja aplikacije s definiranim funkcionalnim zahtjevima.

Testni scenarij	Očekivani rezultat	Dobiveni rezultat
Anonimna prijava korisnika	Korisniku se dodjeljuje UID i otvara unos nadimka	Korisniku je dodijeljen jedinstveni UID te je prikazan dijaloški prozor za unos nadimka.
Ponovno pokretanje aplikacije	Postojeća anonimna prijava ostaje aktivna, a nadimak se ponovno učitava	Nakon ponovnog pokretanja korisnik je ostao prijavljen, a prethodno spremljeni nadimak uspješno je učitao.
Unos praznog nadimka	Korisniku se prikazuje poruka o obveznom unosu nadimka	Aplikacija nije dopustila nastavak te je prikazana poruka o obveznom unosu nadimka.
Skeniranje valjanog QR koda	Prepoznaju se restoran i stol	QR kod uspješno je skeniran, a aplikacija je prepoznala pripadajući restoran i broj stola.
Skeniranje neispravnog QR koda	Prikazuje se poruka o pogrešci	Neispravan format QR koda prepoznat je te je korisniku prikazana poruka o pogrešci.
Skeniranje QR koda s nepostojećim tokenom	Prikazuje se poruka da stol nije pronađen	Token nije pronađen u bazi podataka te je korisniku prikazana poruka da stol ne postoji.
Dohvat digitalnog jelovnika	Prikazuju se dostupna hrana i piće iz baze podataka	Podaci o dostupnoj hrani i piću uspješno su dohvaćeni i prikazani u digitalnom jelovniku.
Slanje narudžbe	Narudžba se sprema u bazu	Potvrđena narudžba uspješno je spremljena u Firebase Realtime Database sa svim pripadajućim podacima.
Otvaranje kategorije proizvoda	Prikazuju se samo proizvodi odabrane kategorije	Nakon odabira kategorije prikazani su samo proizvodi koji joj pripadaju.
Odabir modifikacije proizvoda	Odabrana modifikacija pridružuje se stavci narudžbe	Odabrana modifikacija uspješno je spremljena i prikazana uz odgovarajuću stavku narudžbe.
Unos napomene uz proizvod	Napomena se sprema zajedno sa stavkom narudžbe	Unesena napomena uspješno je spremljena i prikazana uz odabrani proizvod.
Promjena statusa narudžbe	Novi status prikazuje se korisniku bez potrebe za ručnim osvježavanjem	Status je uspješno promijenjen u kuhinjskom dijelu te se bez ručnog osvježavanja prikazao korisniku.

Tablica 5. Rezultati funkcionalnog testiranja mobilne aplikacije. Izvor: izrada autora.

Rezultati funkcionalnog testiranja pokazali su da su sve provjerene funkcionalnosti mobilne aplikacije funkcionirale u skladu s očekivanim ponašanjem. Uspješno su provedeni anonimna prijava i ponovno učitavanje korisničkih podataka, validacija ispravnih i neispravnih QR kodova, dohvat i filtriranje digitalnog jelovnika, prilagodba proizvoda te spremanje narudžbe u Firebase Realtime Database. Potvrđena je i sinkronizacija podataka u stvarnom vremenu jer se promjena statusa izvršena u kuhinjskom dijelu prikazala korisniku bez potrebe za ručnim osvježavanjem zaslona. Testiranje je provedeno ručnim izvođenjem unaprijed definiranih scenarija, zbog čega se rezultati odnose na provjerene slučajeve i ne obuhvaćaju sve moguće kombinacije korisničkih unosa, uređaja i mrežnih uvjeta.

5.2 Evaluacija AI asistenta

Evaluacija AI asistenta provedena je funkcionalnim testiranjem reprezentativnih scenarija koji odgovaraju uobičajenim zahtjevima tijekom naručivanja. Kod preporučenih sustava temeljenih na velikim jezičnim modelima važni su razumijevanje korisničkih zahtjeva, korištenje dostupnog konteksta i generiranje preporuka prilagođenih korisnikovim potrebama [29]. U okviru evaluacije razvijenog sustava provjerene su mogućnosti pretraživanja jelovnika, dodavanja i izmjene proizvoda u košarici, nastavka razgovora, postupnog sastavljanja obroka te postupanja s nejasnim zahtjevima i proizvodima koji nisu dostupni.

Za svaki scenarij provjeravalo se je li sustav ispravno prepoznao korisničku namjeru, vratio valjanu JSON strukturu i omogućio izvršavanje odgovarajuće poslovne logike. Također je provjereno koristi li asistent kontekst prethodnih poruka i trenutni sadržaj košarice te izbjegava li dodavanje proizvoda koji ne postoje u digitalnom jelovniku. Rezultati funkcionalne evaluacije AI asistenta prikazani su u tablici 5.

Testni scenarij	Korisnički zahtjev	Očekivani rezultat	Dobiveni rezultat
Prikaz kategorije proizvoda	„Koje pizze imaš?”	Prikazuju se pizze dostupne u digitalnom jelovniku	Asistent je prepoznao traženu kategoriju i prikazao dostupne pizze.
Dodavanje proizvoda	„Dodaj dvije Coca-Cole.”	Prepoznaju se proizvod i količina te se stavke dodaju u košaricu	Asistent je prepoznao Coca-Colu i količinu dva te dodao proizvode u košaricu.

Dodavanje proizvoda s napomenom	„Dodaj pizzu margaritu bez luka.”	Tražena pizza se dodaje u košaricu s napomenom „bez luka”	Proizvod je dodan u košaricu s korisničkom napomenom.
Prikaz košarice	„Što imam u košarici?”	Prikazuju se trenutačno odabrani proizvodi i njihove količine	Asistent je prikazao sve proizvode koji su se nalazili u košarici.
Postupno sastavljanje obroka	„Želim juhu, tjesteninu i meso.”	Pokreće se postupni odabir proizvoda iz traženih kategorija	Asistent je prepoznao dijelove obroka i korisnika vodio kroz njihov odabir.
Nastavak razgovora	„Dodaj prvi.”	Odabire se prvi proizvod iz prethodno prikazanih prijedloga	Asistent je povezao zahtjev s prethodnim odgovorom i dodao prvi ponuđeni proizvod.
Ponavljanje prethodne narudžbe	„Ponovi moju posljednju narudžbu.”	Dohvaća se prethodna narudžba i traži potvrda prije dodavanja	Asistent je pronašao prethodnu narudžbu i zatražio potvrdu korisnika prije njezina ponovnog dodavanja.
Nepovezan zahtjev	„Kakvo će vrijeme biti sutra?”	Ne izvršava se radnja nad narudžbom, a korisnik se usmjerava na funkcije asistenta	Asistent nije promijenio košaricu i usmjerio je razgovor na jelovnik i naručivanje.
Prepoznavanje alternativnog naziva proizvoda	„Daj mi koka kolu.”	Sustav prepoznaje izraz „koka kola” kao alternativni naziv za proizvod Coca-Cola te ga dodaje u košaricu	Asistent je povezao korisnički izraz s proizvodom Coca-Cola dostupnim u jelovniku i uspješno ga dodao u košaricu.
Zahtjev za nedostupnim proizvodom	„Dodaj sushi.”	Proizvod se ne dodaje u košaricu, a korisnika se obavještava da nije dostupan	Asistent nije dodao proizvod u košaricu te je korisnika obavijestio da proizvod nije pronađen.

Tablica 6. Rezultati funkcionalne evaluacije AI asistenta. Izvor: izrada autora.

Rezultati funkcionalne evaluacije pokazali su da je AI asistent u navedenim scenarijima ispravno prepoznao korisničke namjere i omogućio izvršavanje pripadajuće poslovne logike. Potvrđene su mogućnosti pretraživanja jelovnika, dodavanja proizvoda i napomena, postupnog sastavljanja obroka te nastavka razgovora na temelju prethodnog konteksta. Sustav je uspješno prepoznao i alternativne nazive proizvoda, primjerice izraz „koka kola”, koji je povezan s proizvodom Coca-Cola.

U slučaju nejasnog ili nepovezanog zahtjeva te proizvoda koji nije dostupan u jelovniku sustav nije izvršio neželjenu promjenu nad košaricom. Umjesto toga korisniku je prikazao odgovarajuću poruku, zatražio dodatno pojašnjenje ili ga usmjerio na funkcionalnosti povezane s jelovnikom i naručivanjem.

Evaluacija je provedena ručnim testiranjem unaprijed definiranih scenarija i ne predstavlja statističko istraživanje na većem broju korisnika. Dobiveni rezultati potvrđuju ispravnost ponašanja asistenta u provjerenim slučajevima, ali ne jamče jednako ponašanje za sve moguće formulacije korisničkih zahtjeva.

6 Zaključak

U okviru ovog rada razvijen je sustav za digitalno naručivanje hrane i pića koji objedinjuje mobilnu Android aplikaciju, Firebase servise i AI asistenta. Mobilna aplikacija korisniku omogućuje anonimnu prijavu, skeniranje QR koda stola, pregled jelovnika, prilagodbu proizvoda, izradu narudžbe i praćenje njezina statusa.

Poslužiteljski dio implementiran je s pomoću Firebase Functions servisa, dok se podaci o jelovniku, stolovima, modifikacijama i narudžbama pohranjuju u Firebase Realtime Database bazi podataka. Poslužiteljska strana upravlja komunikacijom s modelom umjetne inteligencije, štiti API ključeve i odvaja poslovnu logiku od mobilne aplikacije.

Posebna vrijednost razvijenog sustava očituje se u primjeni velikog jezičnog modela kao digitalnog konobara koji omogućuje prirodnu komunikaciju tijekom procesa naručivanja. AI asistent u provjerenim scenarijima uspješno je prepoznavao korisničke namjere, pronalazio odgovarajuće proizvode iz jelovnika, odgovarao na pitanja o proizvodima te omogućavao postupno sastavljanje narudžbe kroz razgovor. Time je ostvaren glavni cilj rada – razvoj funkcionalnog sustava koji korisniku pojednostavljuje proces naručivanja i pruža intuitivnije iskustvo korištenja u odnosu na klasično pregledavanje digitalnog jelovnika.

Provedena evaluacija pokazala je da kombinacija velikog jezičnog modela i poslovne logike implementirane na poslužiteljskoj strani omogućuje pouzdanu obradu tipičnih korisničkih zahtjeva. Istodobno je potvrđeno da je za sigurnu primjenu umjetne inteligencije nužno ograničiti njezinu ulogu na razumijevanje korisničkog zahtjeva, dok se sve ključne poslovne odluke i provjere izvršavaju unutar poslužiteljskog dijela sustava.

Unatoč ostvarenim rezultatima, razvijeno rješenje ima određena ograničenja. To se ponajprije odnosi na mogućnost pogrešnog tumačenja dvosmislenih korisničkih zahtjeva, ovisnost o dostupnosti mrežne veze i vanjskih AI servisa te operativne troškove povezane s korištenjem velikih jezičnih modela. Iako su navedena ograničenja ublažena primjenom dodatne poslovne logike i validacije na poslužiteljskoj strani, ona predstavljaju područja za daljnje unapređenje sustava.

U budućem razvoju sustava moguće je implementirati elektroničko plaćanje, korisničke račune i programe vjernosti, višejezičnu podršku, naprednije personalizirane preporuke temeljene na prethodnim narudžbama te dodatno optimizirati AI komponentu smanjenjem potrošnje tokena i vremena odziva. Također, korisno bi bilo provesti opsežniju evaluaciju sa

stvarnim korisnicima kako bi se analiziralo korisničko iskustvo i učinkovitost sustava u stvarnim uvjetima rada.

Razvijeni sustav pokazuje da kombinacija mobilnih tehnologija, računalstva u oblaku i velikih jezičnih modela predstavlja kvalitetnu osnovu za razvoj suvremenih digitalnih rješenja u ugostiteljstvu. Dobiveni rezultati pokazuju da umjetna inteligencija može unaprijediti proces naručivanja kada nadopunjuje jasno definiranu poslovnu logiku. Takva kombinacija pruža ravnotežu između prirodne komunikacije, pouzdanosti sustava i jednostavnosti korištenja.

Literatura

1. Dobartek, *Naruči dostavu hrane online*. [Na internetu]. Dostupno: <https://www.dobartek.hr/> [pristupano: 4.7.2026.]
2. FoodApp, *Naruči hranu online | Dostava i rezervacije*. [Na internetu]. Dostupno: <https://foodapp.hr/> [pristupano: 4.7.2026.]
3. me&u, *Serve – Hospitality Sales and Ordering Tools*. [Na internetu]. Dostupno: <https://www.meandu.com/serve> [pristupano: 4.7.2026.]
4. DoorDash, *Stop Scrolling, Just Ask: A New Conversational Way To Search on DoorDash*. [Na internetu]. Dostupno: <https://about.doordash.com/en-us/news/ask-doordash> [pristupano: 4.7.2026.]
5. Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang i dr., „Recommender Systems in the Era of Large Language Models (LLMs),” *IEEE Transactions on Knowledge and Data Engineering*, sv. 36, br. 11, str. 6889-6907, 2024., doi:10.1109/TKDE.2024.3392335.
6. Google, *Android Studio*, Android Developers. [Na internetu]. Dostupno: <https://developer.android.com/studio> [pristupano: 6.7.2026.]
7. Oracle, *The Java™ Tutorials*. [Na internetu]. Dostupno: <https://docs.oracle.com/javase/tutorial/> [pristupano: 6.7.2026.]
8. Google, *Android Runtime (ART) and Dalvik*, Android Open Source Project. [Na internetu]. Dostupno: <https://source.android.com/docs/core/runtime> [pristupano: 15.7.2026.]
9. J. Bloch, *Effective Java*, 3. izd., Addison-Wesley Professional, 2018.
10. Google, *Firebase Documentation*. [Na internetu]. Dostupno: <https://firebase.google.com/docs> [pristupano: 15.7.2026.]
11. Google, *Firebase Authentication*. [Na internetu]. Dostupno: <https://firebase.google.com/docs/auth> [pristupano: 16.7.2026.]
12. Google, *Firebase Realtime Database*. [Na internetu]. Dostupno: <https://firebase.google.com/docs/database> [pristupano: 16.7.2026.]
13. Google, *Cloud Functions for Firebase*. [Na internetu]. Dostupno: <https://firebase.google.com/docs/functions> [pristupano: 16.7.2026.]
14. H. B. Hassan, S. A. Barakat i Q. I. Sarhan, „Survey on Serverless Computing,” *Journal of Cloud Computing*, sv. 10, čl. 39, 2021., doi: 10.1186/s13677-021-00253-7.
15. Google, *Google Code Scanner (Android only)*. [Na internetu]. Dostupno: <https://developers.google.com/ml-kit/vision/barcode-scanning/code-scanner> [pristupano: 16.7.2026.]
16. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser i I. Polosukhin, „Attention Is All You Need,” u *Advances in Neural Information Processing Systems* 30 (NeurIPS 2017), 2017., str. 5998–6008.
17. OpenAI, *Responses API*, OpenAI Platform Documentation. [Na internetu]. Dostupno: <https://platform.openai.com/docs/api-reference/responses> [pristupano: 17.7.2026.]
18. OpenAI, *Models*, OpenAI Platform Documentation. [Na internetu]. Dostupno:

<https://platform.openai.com/docs/models> [pristupano: 17.7.2026.]

19. J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith i D. C. Schmidt, „A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT,” u Proceedings of the 30th Conference on Pattern Languages of Programs (PLoP 2023), 2023., doi: 10.64346/PLoP2023p05.

20. F. Ricci, L. Rokach i B. Shapira, ur., Recommender Systems Handbook, 3. izd., Springer, 2022.

21. Google, Android Developers Documentation. [Na internetu]. Dostupno: <https://developer.android.com/guide> [pristupano: 20.7.2026.]

22. Google, Platform Architecture, Android Developers. [Na internetu]. Dostupno: <https://developer.android.com/guide/platform> [pristupano: 20.7.2026.]

23. Google, *Activity Lifecycle*, Android Developers. [Na internetu]. Dostupno: <https://developer.android.com/guide/components/activities/activity-lifecycle> [pristupano: 20.7.2026.]

24. A. Dumbravan, Clean Android Architecture: Take a Layered Approach to Writing Clean, Testable, and Decoupled Android Applications, Packt Publishing, 2022.

25. D. Jannach, A. Manzoor, W. Cai i L. Chen, „A Survey on Conversational Recommender Systems,” ACM Computing Surveys, sv. 54, br. 5, čl. 105, 2021., doi: 10.1145/3453154.

26. Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu i dr., „Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys, sv. 55, br. 12, čl. 248, 2023., doi: 10.1145/3571730.

27. OpenAI, GPT-5 mini, OpenAI Platform Documentation. [Na internetu]. Dostupno: <https://developers.openai.com/api/docs/models/gpt-5-mini> [pristupano: 24.8.2026.]

28. P. Leloudas, Introduction to Software Testing: A Practical Guide to Testing, Design, Automation, and Execution, Apress, 2023., doi: 10.1007/978-1-4842-9514-4.

29. J. Wang, Building Recommender Systems Using Large Language Models, Springer, 2025.

Dodatak A – Promptovi korišteni za AI asistenta

U ovom dodatku prikazani su reprezentativni dijelovi promptova korištenih za rad AI asistenta. Radi preglednosti izostavljeni su ponavljajući primjeri i dinamički podaci, poput cjelokupnog digitalnog jelovnika, sadržaja košarice i povijesti razgovora.

A.1 Prompt za planiranje razgovora

Prompt za planiranje razgovora koristi se za prepoznavanje korisničke namjere i određivanje daljnjeg tijeka obrade. Modelu se definiraju njegova uloga, moguće korisničke namjere, pravila obrade i očekivana struktura odgovora.

Ti si AI konobar u restoranu.

Tvoj posao:

- razumjeti korisnikovu poruku
- prepoznati korisničku namjeru
- voditi narudžbu kroz razgovor
- ne izmišljati artikle koji ne postoje u meniju
- vratiti odgovor isključivo u JSON formatu

Moguće namjere uključuju:

- category_question
- add_to_cart
- clarify_order
- show_cart
- recommendation
- repeat_order
- compose_order
- unknown

Očekivani odgovor:

```
{
  "intent": "unknown",
  "category": "",
  "items": [],
  "reply": "",
  "needsClarification": false,
  "confidence": 0.0
}
```

Kontekst:

MENI: [DIGITALNI JELOVNIK]

KOŠARICA: [TRENUTAČNA KOŠARICA]

PRETHODNI RAZGOVOR: [POVIJEST RAZGOVORA]

PORUKA: [KORISNIČKA PORUKA]

A.2 Prompt za obradu narudžbe

Prompt za obradu narudžbe koristi se za izdvajanje proizvoda, količine, modifikacija i napomena iz korisničke poruke. Model smije povezivati korisničke zahtjeve samo s proizvodima i službenim modifikacijama dostupnima u digitalnom jelovniku.

Ti si parser narudžbe za restoran.

Iz korisnikove poruke izvuci artikle, količine i modifikacije.

Vrati SAMO JSON:

```
{
  "items": [
    {
      "name": "TOČAN_NAZIV_IZ_MENIJA",
      "quantity": 1,
      "modifications": [],
      "note": ""
    }
  ],
  "unrecognized": []
}
```

Pravila:

- artikle prepoznaj prema nazivu ili alternativnim nazivima
- vrati samo artikle koji postoje u digitalnom jelovniku
- ne izmišljaj proizvode
- službene modifikacije moraju postojati u podacima artikla
- neslužbene zahtjeve spremi kao napomenu
- ako artikl nije moguće prepoznati, dodaj ga u "unrecognized"
- ako količina nije navedena, koristi vrijednost 1

MODIFIKACIJE:

- Modifikacije smiješ koristiti SAMO iz polja modifications tog artikla.
- Modifikacija mora biti ista kao jedna od ponuđenih vrijednosti iz modifications.
- NIKAD nemoj pretpostavljati modifikacije.
- NIKAD nemoj dodavati slične modifikacije.
- NIKAD nemoj zamijeniti korisnikov zahtjev nekom drugom modifikacijom.
- Ako korisnik traži nešto što nije identična službena modifikacija za taj artikl, stavi to u note.
- Ako korisnik kaže "s kulenom", a artikl nema "S KULENOM" u modifications, modifications mora biti [], a note mora biti "s kulenom".
- Ako modifikacija nije identična jednoj iz modifications, NIKADA je nemoj staviti u modifications, čak ni ako je vrlo slična.

MENI: [DIGITALNI JELOVNIK]

PORUKA: [KORISNIČKA PORUKA]

Popis slika

Slika 1 Arhitektura Android operacijskog sustava	15
Slika 2 Životni ciklus Android aktivnosti	16
Slika 3 UML dijagram slučajeve korištenja razvijenog sustava	18
Slika 4 Struktura mobilne aplikacije	19
Slika 5 Početni zaslon aplikacije	21
Slika 6 Aktivnost za prilagodbu proizvoda	24
Slika 7 Prilagodba proizvoda prije dodavanja u narudžbu	25
Slika 8 Pregled narudžbe prije slanja	26
Slika 9 Zaslon AI asistenta	28
Slika 10 Potvrda uspješno poslane narudžbe	31
Slika 11 Pregled aktivnih narudžbi	32
Slika 12 Praćenje statusa narudžbe	34
Slika 13 Dijaloški prozor za pristup kuhinjskom dijelu aplikacije	36
Slika 14 Pregled aktivnih narudžbi u kuhinjskom dijelu aplikacije	37
Slika 15 Aktivnost za ažuriranje statusa narudžbe	38
Slika 16 Organizacija poslužiteljskog dijela projekta	41
Slika 17 Arhitektura razvijenog sustava i komunikacija između njegovih glavnih komponenti	43
Slika 18 UML sekvencijski dijagram obrade korisničkog zahtjeva	44
Slika 19 Postupno sastavljanje obroka s pomoću AI asistenta	49
Slika 20 Tijek obrade korisničkog zahtjeva unutar poslužiteljskog dijela sustava	51

Popis isječaka programskog koda

Isječak programskog koda 1 Provjera tokena stola u Firebase Realtime Database	22
Isječak programskog koda 2 Slanje narudžbe u Firebase Realtime Database	27
Isječak programskog koda 3 Slanje korisničke poruke AI asistentu	29
Isječak programskog koda 4 Praćenje promjene statusa narudžbe	35
Isječak programskog koda 5 Definiranje prompta za planiranje razgovora	46
Isječak programskog koda 6 Normalizacija strukturiranog plana razgovora	47

Popis tablica

Tablica 1 Usporedba funkcionalnosti analiziranih aplikacija	3
Tablica 2 Usporedba evaluiranih modela na osnovnim scenarijima	54
Tablica 3 Usporedba evaluiranih modela na složenijim scenarijima	56
Tablica 4 Potrošnja tokena i procijenjeni trošak izvođenja scenarija	59
Tablica 5 Rezultati funkcionalnog testiranja mobilne aplikacije	61
Tablica 6 Rezultati funkcionalne evaluacije AI asistenta	63

Sažetak

Cilj ovog diplomskog rada bio je razviti mobilnu aplikaciju za digitalno naručivanje hrane i pića u ugostiteljskim objektima uz primjenu velikih jezičnih modela. Sustav korisniku omogućuje anonimnu prijavu, identifikaciju stola skeniranjem QR koda, pregled digitalnog jelovnika, prilagodbu proizvoda, izradu narudžbe i praćenje njezina statusa. Aplikacija sadrži i kuhinjski dio u kojem osoblje pregledava pristigle narudžbe i ažurira njihove statuse.

Mobilna aplikacija razvijena je u programskom jeziku Java unutar razvojnog okruženja Android Studio. Za autentifikaciju korisnika, pohranu podataka i izvršavanje poslužiteljske logike korišteni su Firebase Authentication, Firebase Realtime Database i Firebase Functions. Google Code Scanner API primijenjen je za skeniranje QR kodova, dok se komunikacija s velikim jezičnim modelom ostvaruje putem OpenAI API-ja.

AI asistent korisniku omogućuje postavljanje pitanja o jelovniku, dobivanje preporuka, dodavanje proizvoda i napomena u košaricu, ponavljanje prethodnih narudžbi te postupno sastavljanje obroka. Veliki jezični model prepoznaje korisničku namjeru, dok se provjera proizvoda i izvršavanje poslovnih pravila provode na poslužiteljskoj strani. Funkcionalno testiranje pokazalo je da su ključne funkcionalnosti mobilne aplikacije i AI asistenta u provjerenim scenarijima radile u skladu s očekivanim ponašanjem.

Ključne riječi: Android, Java, Firebase, OpenAI API, veliki jezični modeli, digitalno naručivanje, AI asistent, mobilna aplikacija

Abstract

The goal of this thesis was to develop a mobile application for the digital ordering of food and beverages in hospitality establishments using large language models. The system enables anonymous user authentication, table identification by scanning a QR code, digital menu browsing, product customization, order creation, and order status tracking. It also includes a kitchen interface through which staff members can review incoming orders and update their statuses.

The mobile application was developed in Java using Android Studio. Firebase Authentication, Firebase Realtime Database, and Firebase Functions were used for user authentication, data storage, and server-side logic. The Google Code Scanner API was used for QR code scanning, while communication with the large language model was implemented through the OpenAI API.

The AI assistant enables users to ask questions about the menu, receive recommendations, add products and notes to the cart, repeat previous orders, and build a meal through conversation. The large language model identifies user intent, while product validation and business rules are handled on the server side. Functional testing showed that the main features of the mobile application and the AI assistant behaved as expected in the evaluated scenarios.

Keywords: Android, Java, Firebase, OpenAI API, large language models, digital ordering, AI assistant, mobile application